1과목 소프트웨어 설계

001 소프트웨어 공학의 기본 원칙

- 현대적인 프로그래밍 기술을 계속적으로 적용해야 한다.
- 개발된 소프트웨어의 품질이 유지되도록 지속적으로 검증해야 한다.
- 소프트웨어 개발 관련 사항 및 결과에 대한 명확한 기록을 유지해야 한다.

002 폭포수 모형

- 이전 단계로 돌아갈 수 없다는 전제하에 각 단계를 확실히 매듭짓고 다음 단계를 진행하는 개발 방법론이다.
- 보헴이 제시한 고전적 생명 주기 모형이다.
- 요구사항을 반영하기 어렵다.

003 프로토타입 모형

- 실제 개발될 소프트웨어에 대한 견본품을 만들어 최종 결과물을 예측하는 모형이다.
- 새로운 요구사항이 도출될 때마다 프로토타입을 새롭게 만든다.
- 요구사항이 불분명할 때 적합한 모형이다.

004 나선형 모형

- 나선을 따라 돌듯이 점진적으로 완벽한 최종 소프트웨어를 개발하는 것이다.
- '계획 수립 → 위험 분석 → 개발 및 검증 → 고객 평가' 과정이 반복적으로 수행된다.

005 애자일 모형의 주요 방법론

- 스크럼(Scrum)
- XP(eXtreme Programming)
- 기능 중심 개발(FDD; Feature Driven Development)
- 칸반(Kanban)
- Lean

006 애자일 개발 4가지 핵심 가치

- 프로세스와 도구보다는 개인과 상호작용에 더 가치를 둔다.
- 방대한 문서보다는 실행되는 SW에 더 가치를 둔다.
- 계약 협상보다는 고객과 협업에 더 가치를 둔다.
- 계획을 따르기 보다는 변화에 반응하는 것에 더 가치를 둔다.

007 스크럼

- 스프린트는 보통 2~4주 정도의 기간 내에서 진행한다.
- 스크럼 회의는 일일 스크럼 회의를 통해 매일 진행한다.

008 XP의 핵심 가치

- 의사소통(Communication)
- 단순성(Simplicity)
- 용기(Courage)
- 존중(Respect)
- 피드백(Feedback)

009 Refactoring(리팩토링)

프로그램 기능의 변경 없이, 시스템의 내부 구조를 재구성한다.

010 주요 비기능 요구사항

- 성능 요구사항
- 보안 요구사항
- 품질 요구사항
- 제약사항
- 인터페이스 요구사항

011 요구사항 개발 프로세스

도출(Elicitation) → 분석(Analysis) → 명세(Specification) → 확인(Validation)

012 요구사항 분석

- 개발 대상에 대한 사용자의 요구사항을 이해하고 문서화(명세화)하는 활동을 의미한다.
- 소프트웨어 개발의 실제적인 첫 단계이다.
- 사용자 요구의 타당성을 조사하고 비용과 일정에 대한 제약을 설정한다
- 사용자의 요구를 정확하게 추출하여 목표를 정하고, 해결 방식을 결정한다.
- 요구사항 분석 도구 : 애자일, UML, 자료 흐름도, 자료 사전, 소단위 명세서, 개체 관계도, 상태 전이도, 제어 명세서 등

013 자료 흐름도의 구성 요소

기호	표기법
프로세스(Process)	물품 확인
자료 흐름(Data Flow)	물품 코드 →
자료 저장소(Data Store)	물품대장
단말(Terminator)	공장

014 HIPO

- 하향식 소프트웨어 개발을 위한 문서화 도구이다.
- 기호, 도표 등을 사용하므로 보기 쉽고 이해하기도 쉽다.
- 기능과 자료의 의존 관계를 동시에 표현할 수 있다.

015 UML

- 시스템 개발자와 고객 또는 개발자 상호 간의 의사소통이 원활하게 이루어지도록 표준화한 대표적인 객체지향 모델링 언어이다.
- 실시간 시스템 및 분산 시스템의 시스템 분석과 설계에 사용이 가능하다.
- 구성 요소 : 사물(Things), 관계(Relationships), 다이어그램(Diagram)

016 UML의 주요 관계

- 일반화(Generalization) 관계 : 하나의 사물이 다른 사물에 비해 더 일반적인지 구체적인지를 표현
- 의존(Dependency) 관계 : 필요에 의해 서로에게 영향을 주는 짧은 시간 동안만 연관을 유지하는 관계를 표현
- 실체화(Realization) 관계 : 사물이 할 수 있거나 해야 하는 기능으로 서로를 그룹화 할 수 있는 관계를 표현

017 구조적(Structural) 다이어그램의 종류

- 클래스 다이어그램(Class Diagram)
- 객체 다이어그램(Object Diagram)
- 컴포넌트 다이어그램(Component Diagram)
- 배치 다이어그램(Deployment Diagram)
- 복합체 구조 다이어그램(Composite Structure Diagram)
- 패키지 다이어그램(Package Diagram)

018 행위(Behavioral) 다이어그램의 종류

- 유스케이스 다이어그램(Use Case Diagram)
- 순차 다이어그램(Sequence Diagram)
- 커뮤니케이션 다이어그램(Communication Diagram)
- 상태 다이어그램(State Diagram)
- 활동 다이어그램(Activity Diagram)
- 상호작용 개요 다이어그램(Interaction Overview Diagram)
- 타이밍 다이어그램(Timing Diagram)

019 스테레오 타입

- UML에서 표현하는 기본 기능 외에 추가적인 기능을 표현하기 위해 사용한다.
- 길러멧(Guilemet)이라고 부르는 겹화살괄호(《 》) 사이에 표현할 형태를 기술한다.

020 유스케이스 다이어그램 - 액터(Actor)

- 시스템과 상호작용을 하는 모든 외부 요소로, 사람이나 외부 시스템을 의미한다.
- 주액터 : 시스템을 사용함으로써 이득을 얻는 대상으로, 주로 사람이 해당함
- 부액터 : 주액터의 목적 달성을 위해 시스템에 서비스를 제공하는 외부 시스템으로, 조직이나 기관 등이 될 수 있음

021 순차(Sequence) 다이어그램의 구성 요소

- 액터(Actor)
- 객체(Object)
- 생명선(Lifeline)
- 실행 상자(Active Box)
- 메시지(Message)

022 사용자 인터페이스의 특징

- 사용자의 편리성과 가독성을 높여준다.
- 작업 시간을 단축시킨다.
- 업무에 대한 이해도를 높여준다.
- 사용자 중심으로 설계되어 있다.

023 사용자 인터페이스의 구분

- CLI(Command Line Interface) : 명령과 출력이 텍스트 형태로 이뤄지는 인터페이스
- GUI(Graphical User Interface) : 아이콘이나 메뉴를 마우스로 선택하여 작업을 수행하는 그래픽 환경의 인터페이스
- NUI(Natural User Interface) : 사용자의 말이나 행동으로 기기를 조작하는 인터페이스

024 사용자 인터페이스의 기본 원칙

- 직관성 : 누구나 쉽게 이해하고 사용할 수 있어야 한다.
- 유효성 : 사용자의 목적을 정확하고 완벽하게 달성해야 한다.
- 학습성 : 누구나 쉽게 배우고 익힐 수 있어야 한다.

025 목업

- 와이어프레임보다 좀 더 실제 화면과 유사하게 만든 정적인 형태의 모형이다.
- 시각적으로만 구성 요소를 배치하는 것으로 실제로 구현되지는 않는다.

026 ISO/IEC 25000

- ISO/IEC 9126과 ISO/IEC 14598을 통합한 소프트웨어 품질 평가 통합 모델 표준이다.
- SQuaRE라고도 한다.

027 ISO/IEC 9126의 품질 특성

- 기능성(Functionality) : 요구사항을 정확하게 만족하는 기능을 제공하는지 여부를 나타냄
  - 하위 특성 : 적절성/적합성, 정밀성/정확성, 상호 운용성, 보안성, 준수성
- 신뢰성(Reliability) : 요구된 기능을 오류 없이 수행할 수 있는 정도를 나타냄
- 사용성(Usability) : 사용자가 쉽게 배우고 사용할 수 있는 정도를 나타냄
- 이식성(Portability) : 다른 환경에서도 얼마나 쉽게 적용할 수 있는지 정도를 나타냄

028 모듈화

- 기능의 분리가 가능하여 인터페이스가 단순해진다.
- 프로그램의 효율적인 관리가 가능하다.
- 오류의 파급 효과를 최소화할 수 있다.
- 모듈의 크기를 너무 작게 나누면 개수가 많아져 모듈간의 통합 비용이 많이 들고, 너무 크게 나누면 개수가 적어 통합 비용은 적게 들지만 모듈 하나의 개발 비용이 많이 든다.

029 추상화의 유형

- 과정 추상화
- 데이터(자료) 추상화
- 제어 추상화

030 하향식 설계

- 프로그램의 상위 모듈에서 하위 모듈 방향으로 설계하는 기법이다.
- 대표적인 하향식 설계 전략으로 단계적 분해가 있다.

031 정보 은닉(Information Hiding)

- 한 모듈 내부에 포함된 절차와 자료들의 정보가 감추어져 다른 모듈이 접근하거나 변경하지 못하도록 하는 기법이다.
- 모듈을 독립적으로 수행할 수 있다.
- 수정, 시험, 유지보수가 용이하다.
- 정보 은닉을 표기할 때 private의 의미는 은닉이다.

032 파이프 - 필터 패턴

- 시스템의 처리 결과물을 파이프를 통해 전달받아 처리한 후 그 결과물을 다시 파이프를 통해 다음 시스템으로 넘겨주는 패턴이다.
- 데이터 변환으로 인한 오버헤드가 발생한다.

033 MVC (Model-View-Controller) 패턴

- 모델(Model) : 서브시스템의 핵심 기능과 데이터를 보관함
- 뷰(View) : 사용자에게 정보를 표시함
- 컨트롤러(Controller) : 사용자로부터 입력된 변경 요청을 처리하기 위해 모델에게 명령을 보냄

034 메시지(Message)

- 객체에게 어떤 행위를 하도록 지시하는 명령 또는 요구사항이다.
- 객체들 간에 상호 작용을 하는 데 사용되는 수단이다.

035 클래스(Class)

- 공통된 속성과 연산(행위)을 갖는 객체의 집합이다.
- 클래스에 속한 각각의 객체를 인스턴스(Instance)라 한다.
- 객체지향 프로그램에서 데이터를 추상화하는 단위이다.

036 캡슐화(Encapsulation)

- 데이터와 데이터를 처리하는 함수를 하나로 묶는 것을 의미한다.
- 외부 모듈의 변경으로 인한 파급 효과가 적다.
- 인터페이스가 단순화된다.
- 재사용이 용이하다.

037 상속(Inheritance)

상위 클래스(부모 클래스)의 모든 속성과 연산을 하위 클래스(자식 클래스)가 물려받는 것이다.

038 다형성(Polymorphism)

- 오버로딩(Overloading) : 메소드의 이름은 같지만 인수를 받는 자료형과 개수를 달리하여 여러 기능을 정의할 수 있음
- 오버라이딩(Overriding) : 메소드의 이름은 같지만 메소드 안의 실행 코드를 달리하여 자식 클래스에서 재정의해서 사용할 수 있음

039 집단화(Aggregation)

클래스들 사이의 '부분-전체(Part-Whole)' 관계 또는 '부분(is-a-part-of)'의 관계로 설명되는 연관성의 한 종류이다.

040 럼바우(Rumbaugh)의 분석 기법

- 객체(Object) 모델링 : 정보 모델링이라고도 하며, 객체들 간의 관계를 규정하여 객체 다이어그램으로 표시하는 것
- 동적(Dynamic) 모델링 : 상태 다이어그램을 이용하여 객체들 간의 동적인 행위를 표현하는 모델링
- 기능(Functional) 모델링 : 자료 흐름도를 이용하여 자료 흐름을 표현한 모델링

041 객체지향 설계 원칙(SOLID 원칙)

- 단일 책임 원칙(SRP; Single Responsibility Principle) : 객체는 단 하나의 책임만 가져야 한다는 원칙
- 개방-폐쇄 원칙(OCP; Open-Closed Principle) : 기존의 코드를 변경하지 않고 기능을 추가할 수 있도록 설계해야 한다는 원칙
- 리스코프 치환 원칙(LSP; Liskov Substitution Principle) : 자식 클래스는 최소한 자신의 부모 클래스에서 가능한 행위는 수행할 수 있어야 한다는 설계 원칙
- 인터페이스 분리 원칙(ISP; Interface Segregation Principle) : 자신이 사용하지 않는 인터페이스와 의존 관계를 맺거나 영향을 받지 않아야 한다는 원칙
- 의존 역전 원칙(DIP; Dependency Inversion Principle) : 각 객체들 간의 의존 관계가 성립될 때, 추상성이 낮은 클래스보다 추상성이 높은 클래스와 의존 관계를 맺어야 한다는 원칙

042 모듈(Module)

- 모듈화를 통해 분리된 시스템의 각 기능들이다.
- 단독으로 컴파일이 가능하다.
- 재사용 할 수 있다.
- 다른 모듈에서의 접근이 가능하다.

043 결합도의 정도(약함 → 강함)

자료 결합도 → 스탬프 결합도 → 제어 결합도 → 외부 결합도 → 공통 결합도 → 내용 결합도

044 결합도의 종류

- 자료(Data) 결합도 : 모듈 간의 인터페이스가 자료 요소로만 구성될 때의 결합도
- 스탬프(Stamp) 결합도 : 배열이나 레코드 등의 자료 구조가 전달될 때의 결합도
- 제어(Control) 결합도 : 제어 신호를 이용하여 통신하거나 제어 요소를 전달하는 결합도
- 외부(External) 결합도 : 데이터(변수)를 외부의 다른 모듈에서 참조할 때의 결합도
- 공통(Common) 결합도 : 공유되는 공통 데이터 영역을 여러 모듈이 사용할 때의 결합도
- 내용(Content) 결합도 : 다른 모듈의 내부 자료를 직접 참조하거나 수정할 때의 결합도

045 응집도의 정도(약함 → 강함)

우연적 응집도 → 논리적 응집도 → 시간적 응집도 → 절차적 응집도 → 교환적 응집도 → 순차적 응집도 → 기능적 응집도

046 주요 응집도

- 절차적(Procedural) 응집도 : 모듈 안의 구성 요소들이 그 기능을 순차적으로 수행할 경우의 응집도
- 시간적(Temporal) 응집도 : 특정 시간에 처리되는 몇 개의 기능을 모아 하나의 모듈로 작성할 경우의 응집도
- 우연적(Coincidental) 응집도 : 각 구성 요소들이 서로 관련 없는 요소로만 구성된 경우의 응집도

047 NS 차트

- 논리의 기술에 중점을 둔 도형을 이용한 표현 방법이다.
- 연속, 선택 및 다중 선택, 반복 등의 제어 논리 구조를 표현한다.
- GOTO나 화살표를 사용하지 않는다.
- 시각적으로 명확히 식별하는 데 적합하다.
- 이해하기 쉽고, 코드 변환이 용이하다.

048 재사용(Reuse)

- 이미 개발된 기능을 새로운 시스템이나 기능 개발에 사용할 수 있는 정도를 의미한다.
- 재사용 규모에 따른 분류 : 함수와 객체, 컴포넌트, 애플리케이션

049 효과적인 모듈 설계 방안

- 결합도는 줄이고 응집도는 높인다.
- 복잡도와 중복성을 줄인다.
- 일관성을 유지시킨다.
- 모듈의 기능은 지나치게 제한적이어서는 안 된다.
- 유지보수가 용이해야 한다.

050 디자인 패턴(Design Pattern)

- 세부적인 구현 방안을 설계할 때 참조할 수 있는 전형적인 해결 방식 또는 예제를 의미한다.
- 디자인 패턴 유형 : 생성 패턴, 구조 패턴, 행위 패턴

051 생성 패턴(Creational Pattern)

- 추상 팩토리(Abstract Factory) : 서로 연관·의존하는 객체들의 그룹으로 생성하여 추상적으로 표현함
- 빌더(Builder) : 작게 분리된 인스턴스를 건축하듯이 조합하여 객체를 생성함
- 팩토리 메소드(Factory Method) : 객체 생성을 서브 클래스에서 처리하도록 분리하여 캡슐화한 패턴으로, 가상 생성자(Virtual Constructor) 패턴이라고도 함
- 프로토타입(Prototype) : 원본 객체를 복제하는 방법으로 객체를 생성함
- 싱글톤(Singleton) : 생성된 객체를 여러 프로세스가 동시에 참조할 수는 없음

052 구조 패턴(Structural Pattern)

- 어댑터(Adapter) : 인터페이스를 다른 클래스가 재사용할 수 있도록 변환함
- 브리지(Bridge) : 서로가 독립적으로 확장할 수 있도록 구성함
- 컴포지트(Composite) : 복합 객체와 단일 객체를 구분 없이 다루고자 할 때 사용함
- 데코레이터(Decorator) : 부가적인 기능을 추가하기 위해 다른 객체들을 덧붙이는 방식으로 구현함
- 퍼싸드(Facade) : 복잡한 서브 클래스들을 피해 더 상위에 인터페이스를 구성함
- 플라이웨이트(Flyweight) : 가능한 한 인스턴스를 공유해서 사용함으로써 메모리를 절약하는 패턴
- 프록시(Proxy) : 접근이 어려운 객체와 여기에 연결하려는 객체 사이에서 인터페이스 역할을 수행하는 패턴

053 행위 패턴 (Behavioral Pattern)

- 책임 연쇄(Chain of Responsibility) : 요청을 한 객체가 처리하지 못하면 다음 객체로 넘어가는 형태
- 커맨드(Command) : 재이용하거나 취소할 수 있도록 요청에 필요한 정보를 저장함
- 인터프리터(Interpreter) : 언어에 문법 표현을 정의함
- 반복자(Iterator) : 접근이 잦은 객체에 대해 동일한 인터페이스를 사용하도록 함
- 중재자(Mediator) : 복잡한 상호 작용을 캡슐화하여 객체로 정의함
- 메멘토(Memento) : 객체를 해당 시점의 상태로 돌릴 수 있는 기능을 제공, Ctrl+Z와 같은 되돌리기 기능을 개발할 때 주로 이용함
- 옵서버(Observer) : 객체에 상속되어 있는 다른 객체들에게 변화된 상태를 전달함
- 상태(State) : 객체의 상태에 따라 동일한 동작을 다르게 처리해야 할 때 사용함
- 전략(Strategy) : 동일한 계열의 알고리즘들을 상호 교환할 수 있게 정의함
- 템플릿 메소드(Template Method) : 하위 클래스에서 세부 처리를 구체화함
- 방문자(Visitor) : 처리 기능을 분리하여 별도의 클래스로 구성함

054 요구사항 검증 방법

- 동료검토(Peer Review) : 작성자가 명세서 내용을 직접 설명하면서 결함을 발견함
- 워크스루(Walk Through) : 미리 배포한 명세서를 사전 검토한 후 결함을 발견함
- 인스펙션(Inspection) : 작성자를 제외한 다른 검토 전문가들이 결함을 발견함
- 동료검토와 워크스루가 비공식적인 검토 방법인데 반해 인스펙션은 공식적인 검토 방법이다.

055 미들웨어(Middleware)

- 분산 컴퓨팅 환경에서 서로 다른 기종 간을 연결한다.
- 운영체제와 응용 프로그램 사이에서 다양한 서비스를 제공한다.
- 위치 투명성을 제공한다.
- 사용자가 미들웨어의 내부 동작을 확인하려면 별도의 응용 소프트웨어를 사용해야 한다.

056 미들웨어의 종류

- DB : 데이터베이스를 연결하기 위한 미들웨어
- RPC : 원격 프로시저를 로컬 프로시저처럼 호출하는 방식의 미들웨어
- MOM : 비동기형 메시지를 전달하는 방식의 미들웨어
- TP-Monitor : 트랜잭션을 처리 및 감시하는 미들웨어
- ORB : 코바 표준 스펙을 구현한 미들웨어
- WAS : 동적인 콘텐츠를 처리하기 위해 사용되는 미들웨어

2과목 소프트웨어 개발

057 자료 구조의 분류

- 선형 구조 : 배열, 선형 리스트, 스택, 큐, 데크
- 비선형 구조 : 트리, 그래프

058 스택(Stack)

- 리스트의 한쪽 끝으로만 자료의 삽입, 삭제 작업이 이루어지는 자료 구조이다.
- 가장 나중에 삽입된 자료가 가장 먼저 삭제되는 후입선출(LIFO) 방식으로 자료를 처리한다.

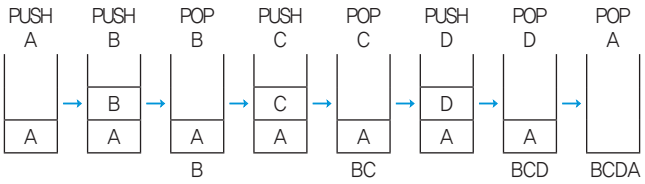
059 스택의 응용 분야

- 인터럽트의 처리
- 수식 계산 및 수식 표기법
- 서브루틴 호출 및 복귀 주소 저장

060 스택의 삽입(Push)과 삭제(Pop)

- PUSH : 스택에 자료를 입력하는 명령
- POP : 스택에서 자료를 출력하는 명령

예제 순서가 A, B, C, D로 정해진 입력 자료를 스택에 입력하였다가 B, C, D, A 순서로 출력하는 과정을 나열하시오.

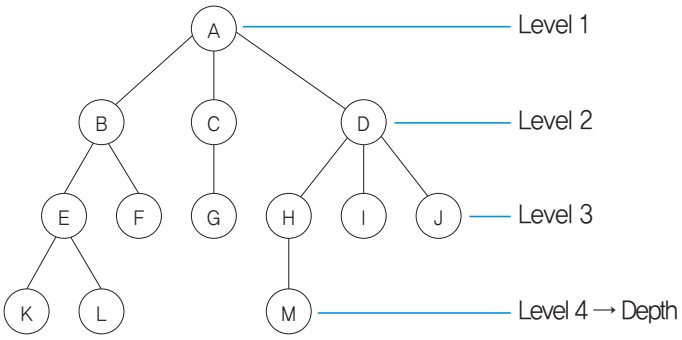


061 방향/무방향 그래프의 최대 간선 수

- 무방향 그래프의 최대 간선 수 : $n(n-1)/2$
- 방향 그래프의 최대 간선 수 : $n(n-1)$

062 트리(Tree)

정점(Node, 노드)과 선분(Branch, 가지)을 이용하여 사이클을 이루지 않도록 구성한 그래프(Graph)의 특수한 형태이다.



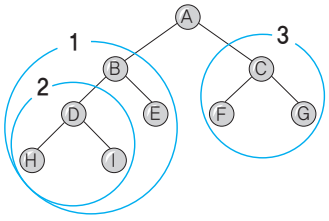
- 디그리(Degree, 차수) : 각 노드에서 뻗어 나온 가지의 수

  예 A = 3, B = 2, C = 1, D = 3
- 단말 노드(Terminal Node) = 잎 노드(Leaf Node) : 자식이 하나도 없는 노드, 즉 디그리가 0인 노드

  예 K, L, F, G, M, I, J

063 이진 트리의 운행법

예 다음 트리를 Inorder, Preorder, Postorder 방법으로 운행했을 때 각 노드를 방문한 순서는?



Preorder 운행법의 방문 순서

① Preorder는 Root → Left → Right이므로 A13이 된다.
② 1은 B2E이므로 AB2E3이 된다.
③ 2는 DHI이므로 ABDHIE3이 된다.
④ 3은 CFG이므로 ABDHIECFG가 된다.
∴ 방문 순서 : ABDHIECFG

Inorder 운행법의 방문 순서

① Inorder는 Left → Root → Right이므로 1A3이 된다.
② 1은 2BE이므로 2BEA3이 된다.
③ 2는 HDI이므로 HDIBEA3이 된다.
④ 3은 FCG이므로 HDIBEAFCG가 된다.
∴ 방문 순서 : HDIBEAFCG

Postorder의 방문 순서

① Postorder는 Left → Right → Root이므로 13A가 된다.
② 1은 2EB이므로 2EB3A가 된다.
③ 2는 HID이므로 HIDEB3A가 된다.
④ 3은 FGC이므로 HIDEBFGCA가 된다.
∴ 방문 순서 : HIDEBFGCA

064 수식의 표기법(Infix → Postfix)

Infix로 표기된 수식에서 연산자를 해당 피연산자 두 개의 뒤(오른쪽)에 오도록 이동하면 Postfix가 된다.

X = A / B * (C + D) + E → X A B / C D + * E + =

① 연산 우선순위에 따라 괄호로 묶는다.
(X = (((A / B) * (C + D)) + E))
② 연산자를 해당 괄호의 뒤로 옮긴다.
X = (((A / B) * (C + D)) + E))
↓
(X (((AB) / (CD) +) * E) +) =
③ 괄호를 제거한다.
X A B / C D + * E + =

065 수식의 표기법(Infix → Prefix)

Infix로 표기된 수식에서 연산자를 해당 피연산자 두 개의 앞(왼쪽)에 오도록 이동하면 Prefix가 된다.

X = A / B * (C + D) + E → X + * / A B + C D E

① 연산 우선순위에 따라 괄호로 묶는다.
(X = (((A / B) * (C + D)) + E))
② 연산자를 해당 괄호의 앞으로 옮긴다.
(X = (((A / B) * (C + D)) + E))
↓
= (X + (* (/ (A B) + (C D)) E))
③ 괄호를 제거한다.
= X + * / A B + C D E

066 수식의 표기법(Postfix → Infix)

Postfix는 Infix 표기법에서 연산자를 해당 피연산자 2개의 뒤(오른쪽)로 이동한 것이므로 연산자를 다시 해당 피연산자 2개의 가운데로 옮기면 된다.

A B C − / D E F + * + → A / (B − C) +D * (E + F)

① 인접한 피연산자 2개와 오른쪽의 연산자를 괄호로 묶는다.
((A (B C −) /) (D (E F +) *) +)
② 연산자를 해당 피연산자의 가운데로 이동시킨다.
((A (B C −) /) (D (E F +) *) +)
↓
((A / (B − C)) + (D * (E + F)))
③ 필요 없는 괄호를 제거한다.
A / (B − C) + D * (E + F)

067 삽입 정렬(Insertion Sort)

예제 8, 5, 6, 2, 4를 삽입 정렬로 정렬하시오.

- 초기 상태 : 8 5 6 2 4
- 1회전 : 8 5 6 2 4 → 5 8 6 2 4

두 번째 값을 첫 번째 값과 비교하여 5를 첫 번째 자리에 삽입하고 8을 한 칸 뒤로 이동시킨다.

- 2회전 : 5 8 6 2 4 → 5 6 8 2 4

세 번째 값을 첫 번째, 두 번째 값과 비교하여 6을 8자리에 삽입하고 8을 한 칸 뒤로 이동시킨다.

5 6 8 2 4 → 2 5 6 8 4

네 번째 값 2를 처음부터 비교하여 맨 처음에 삽입하고 나머지를 한 칸씩 뒤로 이동시킨다.

- 4회전 : 2 5 6 8 4 → 2 4 5 6 8

다섯 번째 값 4를 처음부터 비교하여 5자리에 삽입하고 나머지를 한 칸씩 뒤로 이동시킨다.

068 선택 정렬(Selection Sort)

예제 8, 5, 6, 2, 4를 선택 정렬로 정렬하시오.

- 초기 상태 : 8 5 6 2 4
- 1회전 : 8 5 6 2 4 → 8 5 6 2 4 → 2 5 6 8 4

첫 번째부터 마지막 값 중 최소값 2를 찾아 첫 번째 값 8과 위치를 교환한다.

- 2회전 : 2 5 6 8 4 → 2 5 6 8 4 → 2 4 6 8 5

두 번째부터 마지막 값 중 최소값 4를 찾아 두 번째 값 5와 위치를 교환한다.

- 3회전 : 2 4 6 8 5 → 2 4 6 8 5 → 2 4 5 8 6

세 번째부터 마지막 값 중 최소값 5를 찾아 세 번째 값 6과 위치를 교환한다.

- 4회전 : 2 4 5 8 6 → 2 4 5 8 6 → 2 4 5 6 8

네 번째부터 마지막 값 중 최소값 6을 찾아 네 번째 값 8과 위치를 교환한다.

069 버블 정렬(Bubble Sort)

예제 8, 5, 6, 2, 4를 버블 정렬로 정렬하시오.

- 초기 상태 : 8 5 6 2 4
- 1회전 : 5 8 6 2 4 → 5 6 8 2 4 → 5 6 2 8 4 → 5 6 2 4 8
- 2회전 : 5 6 2 4 8 → 5 2 6 4 8 → 5 2 4 6 8
- 3회전 : 2 5 4 6 8 → 2 4 5 6 8
- 4회전 : 2 4 5 6 8

070 퀵 정렬(Quick Sort)

- 분할(Divide)과 정복(Conquer)을 통해 자료를 정렬한다.
- 피봇(pivot)을 사용하며, 최악의 경우 n(n−1)/2회의 비교를 수행한다.

071 힙 정렬(Heap Sort)

- 완전 이진 트리(Complete Binary Tree)를 이용한 정렬 방식이다.
- 평균과 최악 모두 시간 복잡도는 $O(n\log_2 n)$이다.

072 이분 검색(이진 검색)

- 검색할 데이터가 정렬되어 있어야 한다.
- 중간 레코드 번호(M) : (F+L) / 2 (단, F : 첫 번째 레코드 번호, L : 마지막 레코드 번호)

073 주요 해싱 함수

- 제산법(Division) : 레코드 키 값(K)을 해시표(Hash Table)의 크기보다 큰 수 중에서 가장 작은 소수(Prime, Q)로 나눈 나머지를 홈 주소로 삼는 방식
- 제곱법(Mid-Square) : 레코드 키 값(K)을 제곱한 후 그 중간 부분의 값을 홈 주소로 삼는 방식
- 폴딩법(Folding) : 레코드 키 값(K)을 여러 부분으로 나눈 후 각 부분의 값을 더하거나 XOR한 값을 홈 주소로 삼는 방식
- 숫자 분석법(Digit Analysis) : 키 값을 이루는 숫자의 분포를 분석하여 비교적 고른 자리를 필요한 만큼 선택해서 홈 주소로 삼는 방식

074 체이닝(Chaining)

Collision이 발생하면 버킷에 할당된 연결 리스트(Linked List)에 데이터를 저장하여 문제를 해결하는 방법이다.

075 스키마 3계층

- 외부 스키마 : 사용자나 응용 프로그래머가 각 개인의 입장에서 필요로 하는 데이터베이스의 논리적 구조를 정의한 것
- 개념 스키마 : 데이터베이스의 전체적인 논리적 구조로서, 개체 간의 관계와 제약 조건을 나타내고, 데이터베이스의 접근 권한, 보안 및 무결성 규칙에 관한 명세를 정의함
- 내부 스키마 : 물리적 저장장치의 입장에서 본 데이터베이스 구조로서, 실제로 데이터베이스에 저장될 레코드의 형식을 정의하고 저장 데이터 항목의 표현 방법, 내부 레코드의 물리적 순서 등을 나타냄

076 빌드 자동화 도구

- Ant : 아파치 소프트웨어 재단에서 개발한 소프트웨어
- Maven : Ant의 대안으로 개발한 소프트웨어
- Jenkins : JAVA 기반의 오픈 소스 형태의 빌드 자동화 도구
- Gradle : Groovy를 기반으로 한 오픈 소스 형태의 빌드 자동화 도구

077 소프트웨어 패키징

- 모듈별로 생성한 실행 파일들을 묶어 배포용 설치 파일을 만드는 것이다.
- 개발자가 아니라 사용자를 중심으로 진행한다.

078 DRM(디지털 저작권 관리)의 구성 요소

- 클리어링 하우스(Clearing House) : 저작권에 대한 사용 권한, 라이선스 발급, 사용량에 따른 결제 관리 등을 수행하는 곳
- 콘텐츠 제공자(Contents Provider) : 콘텐츠를 제공하는 저작권자
- 패키저(Packager) : 콘텐츠를 메타 데이터와 함께 배포 가능한 형태로 묶어 암호화하는 프로그램
- 콘텐츠 분배자(Contents Distributor) : 암호화된 콘텐츠를 유통하는 곳이나 사람
- DRM 컨트롤러(DRM Controller) : 배포된 콘텐츠의 이용 권한을 통제하는 프로그램

079 DRM(디지털 저작권 관리)의 기술 요소

- 콘텐츠 암호화 및 키 관리
- 콘텐츠 식별체계 표현
- 라이선스 발급 및 관리
- 정책 관리 기술
- 크랙 방지 기술

080 소프트웨어 설치 매뉴얼

- 설치 매뉴얼은 사용자를 기준으로 작성한다.
- 기본 사항
  - 소프트웨어 개요
  - 설치 관련 파일
  - 프로그램 삭제 등

081 형상 관리(SCM)

- 소프트웨어의 개발 과정에서 소프트웨어의 변경 사항을 관리하기 위해 개발된 일련의 활동이다.
- 목적 : 개발 비용 감소, 방해 요인 최소화
- 관리 항목 : 소스 코드, 프로젝트 분석서, 운영 및 설치 지침서 등
- 형상 관리 도구 : Git, CVS, Subversion 등

082 소프트웨어의 버전 등록 관련 주요 기능

- 체크아웃(Check-Out) : 프로그램을 수정하기 위해 저장소에서 파일을 받아옴
- 체크인(Check-In) : 체크아웃 한 파일의 수정을 완료한 후 저장소의 파일을 새로운 버전으로 갱신함
- 커밋(Commit) : 체크인을 수행할 때 이전에 갱신된 내용이 있는 경우에는 충돌(Conflict)을 알리고 diff 도구를 이용해 수정한 후 갱신을 완료함

083 소프트웨어 버전 관리 도구

RCS, CVS, SVN, BitKeeper

084 파레토 법칙(Pareto Principle)

소프트웨어 테스트에서 오류의 80%는 전체 모듈의 20% 내에서 발견된다는 법칙이다.

085 화이트박스 테스트 (White Box Test)

- 모듈의 원시 코드를 오픈시킨 상태에서 원시 코드의 논리적인 모든 경로를 테스트하여 테스트 케이스를 설계하는 방법이다.
- 프로그램의 제어 구조에 따라 선택, 반복 등의 분기점 부분들을 수행함으로써 논리적 경로를 제어한다.

086 화이트박스 테스트의 종류

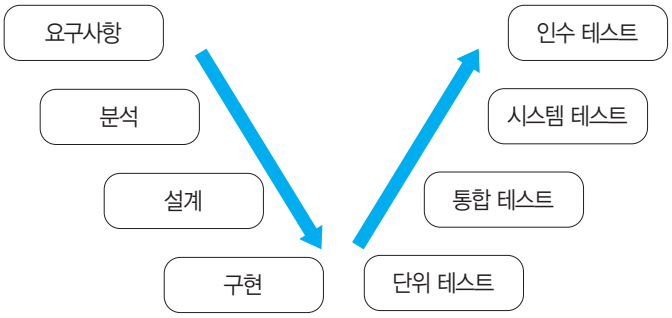
- 기초 경로 검사
- 제어 구조 검사 : 조건 검사, 루프 검사, 데이터 흐름 검사

※ 기초 경로(Base Path = Basis Path) : 수행 가능한 모든 경로를 의미함

087 블랙박스 테스트 종류

- 동치 분할 검사
- 경계값 분석
- 원인-효과 그래프 검사
- 오류 예측 검사
- 비교 검사

088 소프트웨어 생명 주기의 V-모델



089 단위 테스트(Unit Test)

코딩 직후 소프트웨어 설계의 최소 단위인 모듈이나 컴포넌트에 초점을 맞춰 테스트하는 것이다.

090 단위 테스트로 발견 가능한 오류

- 알고리즘 오류에 따른 원치 않는 결과
- 탈출구가 없는 반복문의 사용
- 틀린 계산 수식에 의한 잘못된 결과

091 인수 테스트의 종류

- 알파 테스트 : 개발자의 장소에서 사용자가 개발자 앞에서 행하는 테스트 기법
- 베타 테스트 : 선정된 최종 사용자가 여러 명의 사용자 앞에서 행하는 테스트 기법으로, 필드 테스팅(Field Testing)이라고도 불림

092 통합 테스트 (Integration Test)

- 하향식 통합 테스트 : 프로그램의 상위 모듈에서 하위 모듈 방향으로 통합하면서 테스트하는 기법으로, 깊이 우선 통합법이나 넓이 우선 통합법을 사용함
- 상향식 통합 테스트 : 프로그램의 하위 모듈에서 상위 모듈 방향으로 통합하면서 테스트하는 기법

093 테스트 드라이버(Test Driver)

- 테스트 대상의 하위 모듈을 호출하는 도구이다.
- 매개 변수(Parameter)를 전달하고, 모듈 테스트 수행 후의 결과를 도출한다.
- 상향식 통합 테스트에 사용된다.

094 테스트 스텁(Test Stub)

- 일시적으로 필요한 조건만을 가지고 있는 시험용 모듈이다.
- 하향식 통합 테스트에 사용된다.

095 테스트 오라클(Test Oracle)

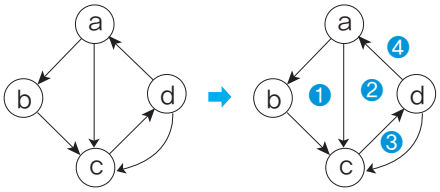
- 테스트 결과가 올바른지 판단하기 위해 사전에 정의된 참 값을 대입하여 비교하는 기법 및 활동이다.
- 종류 : 참 오라클, 샘플링 오라클, 추정 오라클, 일관성 검사 오라클

096 주요 최악의 시간 복잡도

O(1)	입력값(n)에 관계 없이 일정하게 문제 해결에 하나의 단계만을 거침 예 스택의 삽입(Push), 삭제(Pop)
$O(n\log_2 n)$	문제 해결에 필요한 단계가 $n(\log_2 n)$번만큼 수행됨 예 힙 정렬(Heap Sort), 2-Way 합병 정렬(Merge Sort)

097 순환 복잡도

예제 제어 흐름 그래프가 다음과 같을 때 McCabe의 Cyclomatic 수를 계산하시오.



내부 영역 3(❶, ❷, ❸) + 외부 영역 1(❹) = 4

098 클린 코드 작성 원칙

- 가독성 : 누구든지 코드를 쉽게 읽을 수 있도록 작성함
- 단순성 : 코드를 간단하게 작성함
- 의존성 배제 : 코드가 다른 모듈에 미치는 영향을 최소화함
- 중복성 최소화 : 코드의 중복을 최소화함
- 추상화 : 상위 클래스/메소드/함수에서는 간략하게 애플리케이션의 특성을 나타내고, 상세 내용은 하위 클래스/메소드/함수에서 구현함

099 외계인 코드(Alien Code)

아주 오래되거나 참고문서 또는 개발자가 없어 유지 보수 작업이 어려운 코드를 의미한다.

100 소스 코드 품질 분석 도구 - 정적 분석 도구

- 하드웨어 또는 소프트웨어적인 방법으로 코드 분석이 가능하다.
- 종류 : pmd, checkstyle, cppcheck 등

101 EAI의 구축 유형

- Point-to-Point : 애플리케이션을 1:1로 연결함
- Hub & Spoke : 단일 접점인 허브 시스템을 통해 데이터를 전송하는 중앙 집중형 방식
- Message Bus(ESB 방식) : 애플리케이션 사이에 미들웨어를 두어 처리하는 방식
- Hybrid : Hub & Spoke와 Message Bus의 혼합 방식

102 JSON (JavaScript Object Notation)

속성-값 쌍(Attribute-Value Pairs)으로 이루어진 데이터 객체를 전달하기 위해 사람이 읽을 수 있는 텍스트를 사용하는 개방형 표준 포맷이다.

103 AJAX(Asynchronous JavaScript and XML)

- 자바 스크립트(JavaScript) 등을 이용한 비동기 통신 기술이다.
- 클라이언트와 서버 간에 XML 데이터를 교환 및 제어한다.

104 IPSec(IP Security)

- 네트워크 계층에서 IP 패킷 단위의 데이터 변조 방지 및 은닉 기능을 제공하는 프로토콜이다.
- 암호화와 복호화가 모두 가능한 양방향 암호 방식이다.

105 tripwire

크래커가 침입하여 백도어를 만들어 놓거나, 설정 파일을 변경했을 때 분석하는 데이터 무결성 검사 도구 중 하나이다.

106 인터페이스 구현 검증 도구

- xUnit : JUnit, CppUnit, NUnit, HttpUnit 등 다양한 언어에 적용되는 단위 테스트 프레임워크
- STAF : 서비스 호출 및 컴포넌트 재사용 등 다양한 환경을 지원하는 테스트 프레임워크
- FitNesse : 웹 기반 테스트 케이스 설계, 실행, 결과 확인 등을 지원하는 테스트 프레임워크
- NTAF : FitNesse와 STAF의 장점을 통합한 NHN(Naver)의 테스트 자동화 프레임워크
- watir : Ruby를 사용하는 애플리케이션 테스트 프레임워크

3과목 데이터베이스 구축

107 논리적 설계(데이터 모델링)

- 자료를 특정 DBMS가 지원하는 논리적 자료 구조로 변환(mapping)시키는 과정이다.
- 트랜잭션의 인터페이스를 설계한다.
- 개념 스키마를 평가 및 정제한다.

108 물리적 설계

- 논리적 구조로 표현된 데이터를 물리적 구조의 데이터로 변환하는 과정이다.
- 데이터베이스 파일의 저장 구조 및 액세스 경로를 결정한다.
- 저장 레코드의 형식, 순서, 접근 경로, 조회가 집중되는 레코드와 같은 정보를 사용한다.

109 관계의 형태 - 일 대 다(1:N)

개체 집합 A의 각 원소는 개체 집합 B의 원소 여러 개와 대응하고 있지만, 개체 집합 B의 각 원소는 개체 집합 A의 원소 한 개와 대응하는 관계이다.

110 데이터 모델에 표시할 요소

- 구조(Structure) : 논리적으로 표현된 개체 타입들 간의 관계로서 데이터 구조 및 정적 성질을 표현함
- 연산(Operation) : 데이터베이스에 저장된 실제 데이터를 처리하는 작업에 대한 명세로서 데이터베이스를 조작하는 기본 도구
- 제약 조건(Constraint) : 데이터베이스에 저장될 수 있는 실제 데이터의 논리적인 제약 조건

111 E-R 다이어그램

기호	기호 이름	의미
▭	사각형	개체(Entity) 타입
◇	마름모	관계(Relationship) 타입
⬭	타원	속성(Attribute)
⬭	이중 타원	다중값 속성(복합 속성)
—	선, 링크	개체 타입과 속성을 연결

112 튜플(Tuple)

- 릴레이션을 구성하는 각각의 행을 말한다.
- 튜플의 수 = 카디널리티(Cardinality)

113 속성(Attribute)

- 데이터베이스를 구성하는 가장 작은 논리적 단위이다.
- 속성의 수 = 디그리(Degree) = 차수

114 도메인(Domain)

하나의 애트리뷰트가 취할 수 있는 같은 타입의 원자(Atomic)값들의 집합이다.

115 릴레이션의 특징

- 한 릴레이션에는 똑같은 튜플이 포함될 수 없으므로 릴레이션에 포함된 튜플들은 모두 상이하다.
- 한 릴레이션에 포함된 튜플 사이에는 순서가 없다.
- 속성의 유일한 식별을 위해 속성의 명칭은 유일해야 한다.
- 속성의 값은 논리적으로 더 이상 쪼갤 수 없는 원자값만을 저장한다.

116 후보키(Candidate Key)

- 릴레이션을 구성하는 속성들 중에서 튜플을 유일하게 식별하기 위해 사용하는 속성들의 부분집합, 즉 기본키로 사용할 수 있는 속성들을 말한다.
- 릴레이션에 있는 모든 튜플에 대해서 유일성과 최소성을 만족시켜야 한다.

117 슈퍼키(Super Key)

- 한 릴레이션 내에 있는 속성들의 집합으로 구성된 키이다.
- 릴레이션을 구성하는 모든 튜플에 대해 유일성은 만족시키지만, 최소성은 만족시키지 못한다.

118 외래키(Foreign Key)

- 다른 릴레이션의 기본키를 참조하는 속성 또는 속성들의 집합을 의미한다.
- 한 릴레이션에 속한 속성 A와 참조 릴레이션의 기본키인 B가 동일한 도메인 상에서 정의되었을 때의 속성 A를 외래키라고 한다.

119 무결성

- 개체 무결성 : 기본 테이블의 기본키를 구성하는 어떤 속성도 Null 값이나 중복값을 가질 수 없다는 규정
- 참조 무결성 : 외래키 값은 Null이거나 참조 릴레이션의 기본키 값과 동일해야 함. 즉 릴레이션은 참조할 수 없는 외래키 값을 가질 수 없다는 규정

120 관계대수

- 관계형 데이터베이스에서 원하는 정보와 그 정보를 검색하기 위해서 어떻게 유도하는가를 기술하는 절차적인 언어이다.
- 질의에 대한 해를 구하기 위해 수행해야 할 연산의 순서를 명시한다.

121 순수 관계 연산자 - Select

- 릴레이션에 존재하는 튜플 중에서 선택 조건을 만족하는 튜플의 부분집합을 구하여 새로운 릴레이션을 만드는 연산이다.
- 기호 : 시그마(σ)

122 순수 관계 연산자 - Project

- 주어진 릴레이션에서 속성 리스트에 제시된 속성 값만을 추출하여 새로운 릴레이션을 만드는 연산이다.
- 기호 : 파이(π)

123 순수 관계 연산자 - Join

- 공통 속성을 중심으로 두 개의 릴레이션을 하나로 합쳐서 새로운 릴레이션을 만드는 연산이다.
- 기호 : ⋈

124 순수 관계 연산자 - Division

- X⊃Y인 두 개의 릴레이션 R(X)와 S(Y)가 있을 때, R의 속성이 S의 속성값을 모두 가진 튜플에서 S가 가진 속성을 제외한 속성만을 구하는 연산이다.
- 기호 : ÷

125 일반 집합 연산자 - 교차곱 (CARTESIAN PRODUCT)

- 교차곱의 디그리는 두 릴레이션의 디그리를 더한 것과 같다.
- 교차곱의 카디널리티는 두 릴레이션의 카디널리티를 곱한 것과 같다.

126 관계해석

- 관계 데이터 모델의 제안자인 코드(Codd)가 수학의 Predicate Calculus(술어 해석)에 기반을 두고 관계 데이터베이스를 위해 제안했다.
- 관계해석은 원하는 정보가 무엇이라는 것만 정의하는 비절차적 특성을 지닌다.

127 정규화(Normalization)

- 함수적 종속성 등의 종속성 이론을 이용하여 잘못 설계된 관계형 스키마를 더 작은 속성의 세트로 쪼개어 바람직한 스키마로 만들어 가는 과정이다.
- 데이터 중복을 배제하여 이상(Anomaly)의 발생 방지한다.

128 이상(Anomaly)

- 정규화를 거치지 않으면 데이터베이스 내에 데이터들이 불필요하게 중복되어 릴레이션 조작 시 예기치 못한 곤란한 현상이 발생하는 것을 의미한다.
- 종류 : 삽입 이상, 삭제 이상, 갱신 이상

129 정규화 과정

비정규 릴레이션
↓ 도메인이 원자값
1NF
↓ 부분적 함수 종속 제거
2NF
↓ 이행적 함수 종속 제거
3NF
↓ 결정자이면서 후보키가 아닌 것 제거
BCNF
↓ 다치 종속 제거
4NF
↓ 조인 종속성 이용
5NF

130 함수적 종속 (Functional Dependency)

- 데이터들이 어떤 기준값에 의해 종속되는 것을 의미한다.
- '학번'에 따라 '이름'이 결정될 때 '이름'을 '학번'에 함수 종속적이라고 하며 '학번 → 이름'과 같이 쓴다.

131 이행적 종속 관계

A → B이고 B → C일 때 A → C를 만족하는 관계를 의미한다.

132 반정규화(Denormalization)

시스템의 성능 향상, 개발 및 운영의 편의성 등을 위해 정규화된 데이터 모델을 통합, 중복, 분리하는 과정으로, 의도적으로 정규화 원칙을 위배하는 행위이다.

133 시스템 카탈로그 (System Catalog)

- 시스템 그 자체에 관련이 있는 다양한 객체에 관한 정보를 포함하는 시스템 데이터베이스이다.
- 사용자가 시스템 카탈로그 내용을 검색할 수는 있지만 갱신할 수는 없다.

134 트랜잭션의 특성

- Atomicity(원자성) : 트랜잭션의 연산은 데이터베이스에 모두 반영되도록 완료(Commit)되든지 아니면 전혀 반영되지 않도록 복구(Rollback)되어야 함
- Consistency(일관성) : 트랜잭션이 그 실행을 성공적으로 완료하면 언제나 일관성 있는 데이터베이스 상태로 변환함
- Isolation(독립성) : 둘 이상의 트랜잭션이 동시에 병행 실행되는 경우 어느 하나의 트랜잭션 실행 중에 다른 트랜잭션의 연산이 끼어들 수 없음
- Durability(영속성) : 성공적으로 완료된 트랜잭션의 결과는 시스템이 고장나더라도 영구적으로 반영되어야 함

135 인덱스(Index)

- 데이터 레코드를 빠르게 접근하기 위해 〈키 값, 포인터〉 쌍으로 구성되는 데이터 구조이다.
- 데이터 정의어(DDL)를 이용하여 사용자가 생성, 변경, 제거할 수 있다.

136 뷰(View)

- 기본 테이블로부터 유도된, 이름을 가지는 가상 테이블이다.
- 뷰는 가상 테이블이기 때문에 물리적으로 구현되어 있지 않다.
- 뷰로 구성된 내용에 대한 삽입, 삭제, 갱신 연산에 제약이 따른다.
- 뷰를 정의할 때는 CREATE문, 제거할 때는 DROP문을 사용한다.

137 파티션의 종류

- 범위 분할(Range Partitioning) : 지정한 열의 값을 기준으로 범위를 지정하여 분할함
  예 일별, 월별, 분기별 등
- 해시 분할(Hash Partitioning) : 해시 함수를 적용한 결과값에 따라 데이터를 분할함
- 조합 분할(Composite Partitioning) : 범위 분할로 분할한 다음 해시 함수를 적용하여 다시 분할하는 방식
- 목록 분할(List Partitioning) : 지정한 열 값에 대한 목록을 만들어 이를 기준으로 분할함
- 라운드 로빈 분할(Round Robin Partitioning) : 레코드를 균일하게 분배하는 방식

138 분산 데이터베이스의 목표

- 위치 투명성(Location Transparency) : 액세스하려는 데이터베이스의 실제 위치를 알 필요 없이 단지 데이터베이스의 논리적인 명칭만으로 액세스할 수 있음
- 중복 투명성(Replication Transparency) : 동일 데이터가 여러 곳에 중복되어 있더라도 사용자는 마치 하나의 데이터만 존재하는 것처럼 사용하고, 시스템은 자동으로 여러 자료에 대한 작업을 수행함
- 병행 투명성(Concurrency Transparency) : 분산 데이터베이스와 관련된 다수의 트랜잭션들이 동시에 실현되더라도 그 트랜잭션의 결과는 영향을 받지 않음
- 장애 투명성(Failure Transparency) : 트랜잭션, DBMS, 네트워크, 컴퓨터 장애에도 불구하고 트랜잭션을 정확하게 처리함

139 분할(Fragment)

- 수평 분할 : 특정 속성의 값을 기준으로 행(Row) 단위로 분할
- 수직 분할 : 데이터 컬럼(속성) 단위로 분할

140 접근통제 기술

- 임의 접근통제(DAC; Discretionary Access Control) : 데이터에 접근하는 사용자의 신원에 따라 접근 권한을 부여하는 방식
- 강제 접근통제(MAC; Mandatory Access Control) : 주체와 객체의 등급을 비교하여 접근 권한을 부여하는 방식
- 역할기반 접근통제(RBAC; Role Based Access Control) : 사용자의 역할에 따라 접근 권한을 부여하는 방식

141 DAS (Direct Attached Storage)

- 서버와 저장장치를 전용 케이블로 직접 연결하는 방식이다.
- DAS에서 사용되는 프로토콜에는 SATA, eSATA, SCSI, SAS 등이 있다.

142 SAN(Storage Area Network)

- DAS의 빠른 처리와 NAS의 파일 공유 장점을 혼합한 방식이다.
- SAN에서 사용되는 프로토콜에는 iSCSI, FC, FCoE 등이 있다.

143 DDL(데이터 정의어)

- 스키마, 도메인, 테이블, 뷰, 인덱스를 정의하거나 변경 또는 삭제할 때 사용하는 언어이다.
- CREATE : 스키마, 도메인, 테이블, 뷰, 인덱스를 정의함
- ALTER : TABLE에 대한 정의를 변경하는 데 사용함
- DROP : 스키마, 도메인, 테이블, 뷰, 인덱스를 삭제함

144 DML(데이터 조작어)

- 데이터베이스 사용자가 응용 프로그램이나 질의어를 통하여 저장된 데이터를 실질적으로 처리하는 데 사용되는 언어이다.
- SELECT : 테이블에서 조건에 맞는 튜플을 검색함
- INSERT : 테이블에 새로운 튜플을 삽입함
- DELETE : 테이블에서 조건에 맞는 튜플을 삭제함
- UPDATE : 테이블에서 조건에 맞는 튜플의 내용을 변경함

145 DCL(데이터 제어어)

- 데이터의 보안, 무결성, 회복, 병행 수행 제어 등을 정의하는 데 사용되는 언어이다.
- COMMIT : 명령에 의해 수행된 결과를 실제 물리적 디스크로 저장하고, 데이터베이스 조작 작업이 정상적으로 완료되었음을 관리자에게 알려줌
- ROLLBACK : 데이터베이스 조작 작업이 비정상적으로 종료되었을 때 원래의 상태로 복구함
- GRANT : 데이터베이스 사용자에게 사용 권한을 부여함
- REVOKE : 데이터베이스 사용자의 사용 권한을 취소함

146 CREATE TABLE

- 테이블을 정의하는 명령문이다.
- 표기 형식

```
CREATE TABLE 테이블명
        (속성명 데이터_타입 [DEFAULT 기본값] [NOT NULL], …
        [, PRIMARY KEY(기본키_속성명, …)]
        [, UNIQUE(대체키_속성명, …)]
        [, FOREIGN KEY(외래키_속성명, …)]
            [REFERENCES 참조테이블(기본키_속성명, …)]
            [ON DELETE 옵션]
            [ON UPDATE 옵션]
        [, CONSTRAINT 제약조건명] [CHECK (조건식)]);
```

147 ALTER TABLE

- 테이블에 대한 정의를 변경하는 명령문이다.
- 표기 형식

```
ALTER TABLE 테이블명 ADD 속성명 데이터_타입 [DEFAULT '기본값'];
ALTER TABLE 테이블명 ALTER 속성명 [SET DEFAULT '기본값'];
ALTER TABLE 테이블명 DROP COLUMN 속성명 [CASCADE];
```

148 DROP TABLE

- 기본 테이블을 제거하는 명령문이다.
- 표기 형식

```
DROP TABLE 테이블명 [CASCADE | RESTRICT];
```

- CASCADE : 제거할 요소를 참조하는 다른 모든 개체를 함께 제거함
- RESTRICT : 다른 개체가 제거할 요소를 참조중일 때는 제거를 취소함

149 삽입문(INSERT INTO~)

- 기본 테이블에 새로운 튜플을 삽입할 때 사용한다.
- 표기 형식

```
INSERT INTO 테이블명([속성명1, 속성명2,…])
VALUES (데이터1, 데이터2,… );
```

150 삭제문(DELETE FROM~)

- 기본 테이블에 있는 튜플들 중에서 특정 튜플(행)을 삭제할 때 사용한다.
- 표기 형식

```
DELETE
FROM 테이블명
[WHERE 조건];
```

151 갱신문(UPDATE~ SET~)

- 기본 테이블에 있는 튜플들 중에서 특정 튜플의 내용을 변경할 때 사용한다.
- 표기 형식

```
UPDATE 테이블명
SET 속성명 = 데이터[, 속성명=데이터, …]
[WHERE 조건];
```

152 데이터 조작문의 네 가지 유형

- SELECT(검색) : SELECT~ FROM~ WHERE~
- INSERT(삽입) : INSERT INTO~ VALUES~
- DELETE(삭제) : DELETE~ FROM~ WHERE~
- UPDATE(변경) : UPDATE~ SET~ WHERE~

153 Select문

```
SELECT [PREDICATE] [테이블명.]속성명1, [테이블명.]속성명2,…
FROM 테이블명1, 테이블명2,…
[WHERE 조건]
[GROUP BY 속성명1, 속성명2,…]
[HAVING 조건]
[ORDER BY 속성명 [ASC | DESC]];
```

- SELECT절
  - Predicate : 불러올 튜플 수를 제한할 명령어
    - ▶ DISTINCT : 중복된 튜플이 있으면 그 중 첫 번째 한 개만 검색
  - 속성명 : 검색하여 불러올 속성(열) 및 수식들
- FROM절 : 질의에 의해 검색될 데이터들을 포함하는 테이블명
- WHERE절 : 검색할 조건
- GROUP BY절 : 특정 속성을 기준으로 그룹화하여 검색할 때 그룹화 할 속성
- HAVING절 : 그룹에 대한 조건
- ORDER BY절
  - 속성명 : 정렬의 기준이 되는 속성명
  - [ASC | DESC] : 정렬 방식(ASC는 오름차순, DESC 또는 생략하면 내림차순)

154 조건 연산자 - LIKE

- 대표 문자를 이용해 지정된 속성의 값이 문자 패턴과 일치하는 튜플을 검색하기 위해 사용된다.
- 대표 문자
  - % : 모든 문자를 대표함
  - _ : 문자 하나를 대표함
  - # : 숫자 하나를 대표함

155 조건 연산자 - BETWEEN

지정된 속성이 두 숫자 사이의 값을 가지는 튜플을 검색하기 위해 사용된다.

예 생일이 '01/09/69'에서 '10/22/73' 사이인 자료만 검색

→ WHERE 생일 BETWEEN #01/09/69# AND #10/22/73#

156 하위 질의

조건절에 주어진 질의를 먼저 수행하여 그 검색 결과를 조건절의 피연산자로 사용한다.

예 '취미'가 "나이트댄스"인 사원의 '이름'과 '주소'를 검색하시오.

```
SELECT 이름, 주소
FROM 사원
WHERE 이름 = (SELECT 이름 FROM 여가활동 WHERE 취미 = '나이트댄스');
```

157 그룹 함수

- GROUP BY절에 지정된 그룹별로 속성의 값을 집계할 때 사용된다.
- COUNT/SUM/AVG/MAX/MIN(속성명) : 그룹별 튜플 수/합계/평균/최대값/최소값을 구하는 함수

158 집합 연산자의 종류

- UNION : 두 조회 결과를 통합하여 모두 출력하되, 중복된 행은 한 번만 출력함
- UNION ALL : 두 조회 결과를 통합하여 모두 출력하되, 중복된 행도 그대로 출력함
- INTERSECT : 두 조회 결과 중 공통된 행만 출력함
- EXCEPT : 첫 번째 조회 결과에서 두 번째 조회 결과를 제외한 행을 출력함

159 트리거(Trigger)

데이터의 삽입(Insert), 갱신(Update), 삭제(Delete) 등의 이벤트(Event)가 발생할 때마다 관련 작업이 자동으로 수행되는 절차형 SQL이다.

4과목 프로그래밍 언어 활용

160 C/JAVA의 자료형

종류	C	JAVA
문자	char(1Byte)	char(1Byte)
정수	int(4Byte)	int(4Byte)
	long(8Byte)	long(8Byte)
논리		boolean(1Byte)

161 C언어의 구조체

- 자료의 종류가 다른 변수의 모임이다.
- 예약어 struct를 이용해 정의한다.

162 Python의 시퀀스 자료형

- 리스트(List) : 필요에 따라 개수를 늘리거나 줄일 수 있음
- 튜플(Tuple) : 요소의 추가, 삭제, 변경은 불가능함
- range : 연속된 숫자를 생성함

163 변수명 작성 규칙

- 영문자, 숫자, _(under bar)를 사용할 수 있다.
- 첫 글자는 숫자는 올 수 없다.
- 공백이나 *, +, –, / 등의 특수문자를 사용할 수 없다.
- 대·소문자를 구분한다.
- 예약어를 변수명으로 사용할 수 없다.

164 가비지 콜렉터 (Garbage Collector)

선언만 하고 사용하지 않는 변수들이 점유한 메모리 공간을 강제로 해제하여 다른 프로그램들이 사용할 수 있도록 하는 것이다.

165 산술 연산자

연산자	의미	비고
%	나머지	정수만 연산할 수 있으며, 실수를 사용하면 오류가 발생함
++	증가	• **전치** : 변수 앞에 증감 연산자가 오는 형태로 먼저 변수의 값을 증감시킨 후 변수를 연산에 사용함(++a, --a)
--	감소	• **후치** : 변수 뒤에 증감 연산자가 오는 형태로 먼저 변수를 연산에 사용한 후 변수의 값을 증감시킴(a++, a--)

166 비트 연산자

- & (and) : 모든 비트가 1일 때만 1
- ^ (xor) : 모든 비트가 같으면 0, 하나라도 다르면 1
- | (or) : 모든 비트 중 한 비트라도 1이면 1
- ~ (not) : 각 비트의 부정, 0이면 1, 1이면 0
- 〈〈 (왼쪽 시프트) : 비트를 왼쪽으로 이동
- 〉〉 (오른쪽 시프트) : 비트를 오른쪽으로 이동

167 논리 연산자

- ! (not) : 부정
- && (and) : 모두 참이면 참
- || (or) : 하나라도 참이면 참

168 조건 연산자

조건에 따라 서로 다른 수식을 수행한다.

예 mx = a 〈 b ? b : a;

a가 b보다 작으면 mx에 b를 저장하고 그렇지 않으면 mx에 a를 저장한다.

169 연산자 우선순위

대분류	중분류	연산자	결합 규칙	우선 순위
단항 연산자	단항 연산자	! ~ ++ -- sizeof	←	높음 ↑
이항 연산자	산술 연산자	* / %	→	
		+ -		
	시프트 연산자	〈〈 〉〉		
	관계 연산자	〈 〈= 〉= 〉		
		== !=		
	비트 연산자	& ^ |		
	논리 연산자	&& ||		
삼항 연산자	조건 연산자	? :	→	
대입 연산자	대입 연산자	= += -= *= /= %= 〈〈= 〉〉= 등	←	↓
순서 연산자	순서 연산자	,	→	낮음

170 주요 서식 문자열

- %d : 정수형 10진수를 입·출력하기 위해 지정함
- %c : 문자를 입·출력하기 위해 지정함
- %s : 문자열을 입·출력하기 위해 지정함

171 printf( ) 함수

인수로 주어진 값을 화면에 출력하는 함수이다.

예 printf("%d, %c", a, b);

a의 값을 정수로 출력하고 쉼표(,)와 공백 한 칸을 띄운 후, b의 값을 문자로 출력한다.

172 JAVA의 출력 함수

- printf( )

  예 System.out.printf("%d", r);

  r의 값을 10진수 정수로 출력한다.

- print( )

  예 System.out.print(r + s);

  r과 s를 더한 값을 출력한다.

- println( )

  예 System.out.println(r + "은(는) 소수");

  r의 값과 **은(는) 소수**를 출력한 후, 커서를 다음 줄의 처음으로 옮긴다.

173 단순 if문

- 조건이 한 개일 때 사용하는 제어문이다.
- 조건이 참일 때만 실행하는 경우

예

```
if (a > b)
    printf("Gilbut");
```

a가 b보다 크면 Gilbut을 출력하고, 아니면 if문을 벗어난다.

- 조건이 참일 때와 거짓일 때 실행할 문장이 다른 경우

예

```
if (a > b)
    printf("참");
else
    printf("거짓");
```

a가 b보다 크면 **참**을 출력하고, 아니면 **거짓**을 출력한다.

174 switch문

- 조건에 따라 분기할 곳이 여러 곳인 경우 간단하게 처리할 수 있는 제어문이다.
- break문이 생략되면 수식과 레이블이 일치할 때 실행할 문장부터 break문 또는 switch문이 종료될 때까지 모든 문장이 실행된다.

예

```
switch(a) {
case 1:
    printf("바나나");
    break;
case 2:
    printf("딸기");
    break;
default:
    printf("없음");
}
```

– a가 1이면 바나나를 출력하고 switch문을 탈출한다.
– a가 2면 딸기를 출력하고 switch문을 탈출한다.
– a가 1이나 2가 아니면 없음을 출력하고 switch문을 탈출한다.

175 for문

초기값, 최종값, 증가값을 지정하는 수식을 이용해 정해진 횟수를 반복하는 제어문이다.

예

```
for (i = 1; i <= 10 ; i++)
    sum = sum + i;
```

반복 변수 i가 1부터 1씩 증가하면서 10보다 작거나 같은 동안 sum에 i의 값을 누적시킨다.

176 while문

조건이 참인 동안 실행할 문장을 반복 수행하는 제어문이다.

예
```
while (i <= 10)
    i = i + 1;
```

i가 10보다 작거나 같은 동안 i의 값을 1씩 누적시킨다.

177 do~while문

- 조건이 참인 동안 정해진 문장을 반복 수행하다가 조건이 거짓이면 반복문을 벗어난다.
- 실행할 문장을 무조건 한 번 실행한 다음 조건을 판단하여 탈출 여부를 결정한다.

예
```
do
    i = i + 1;
while (i <= 10);
```

i가 10보다 작거나 같은 동안 i의 값을 1씩 누적시킨다.

178 1차원 배열

변수들을 일직선상의 개념으로 조합한 배열이다.

예 char a[3] = {'A', 'B', 'C'};

3개의 요소를 갖는 문자형 배열 a를 선언한다.

	a[0]	a[1]	a[2]
배열 a	'A'	'B'	'C'

179 2차원 배열

변수들을 평면, 즉 행과 열로 조합한 배열이다.

예 int b[2][3] = {{11, 22, 33}, {44, 55, 66}};

2개의 행과 3개의 열을 갖는 정수형 배열 b를 선언한다.

	a[0][0]	a[0][1]	a[0][2]
배열 b	11	22	33
	44	55	66
	a[1][0]	a[1][1]	a[1][2]

180 배열 형태의 문자열 변수

- C언어에서는 큰따옴표(" ")로 묶인 글자는 글자 수에 관계없이 문자열로 처리된다.
- 배열에 문자열을 저장하면 문자열의 끝을 알리기 위한 널 문자('\0')가 문자열 끝에 자동으로 삽입된다.

예 char a[5] = "love";

5개의 요소를 갖는 문자형 배열 a를 선언하고, "love"로 초기화한다.

배열 a	'l'	'o'	'v'	'e'	'\0'
	a[0]	a[1]	a[2]	a[3]	a[4]

181 포인터와 포인터 변수

- 포인터 변수를 선언할 때는 자료의 형을 먼저 쓰고 변수명 앞에 간접 연산자 *를 붙인다(예 int *a;).
- 포인터 변수에 주소를 저장하기 위해 변수의 주소를 알아낼 때는 변수 앞에 번지 연산자 &를 붙인다(예 a = &b;).
- 실행문에서 포인터 변수에 간접 연산자 *를 붙이면 해당 포인터 변수가 가리키는 곳의 값을 말한다(예 c = *a;).

예제 다음 C언어로 구현된 프로그램의 출력 결과를 확인하시오.

```
main( )
{
 int a = 50; ❶
 int *b; ❷
 b = &a; ❸
 *b = *b+20; ❹
 printf("%d, %d", a, *b); ❺
}
```

❶ 정수형 변수 a를 선언하고 50으로 초기화한다.
❷ 정수형 변수가 저장된 곳의 주소를 기억할 포인터 변수 b를 선언한다.
❸ 정수형 변수 a의 주소를 포인터 변수 b에 기억시킨다. b에는 a의 주소가 저장된다.
❹ b가 가리키는 곳의 값에 20을 더한다. b가 가리키는 곳이 a이므로 결국 a의 값도 바뀌는 것이다.
❺ 결과 70, 70

182 초치기 포인터와 배열

- 배열을 포인터 변수에 저장한 후 포인터를 이용해 배열의 요소에 접근할 수 있다.
- 배열 위치를 나타내는 첨자를 생략하고 배열의 대표명만 지정하면 배열의 첫 번째 요소의 주소를 지정하는 것과 같다.

예

```
❶ int a[5], *b
❷ b = a;
❸ b = &a[0];
```

❶ 5개의 요소를 갖는 정수형 배열 a와 정수형 포인터 변수 b를 선언한다.
❷ 배열의 대표명을 적었으므로 a 배열의 시작 주소인 a[0]의 주소를 b에 저장한다.
❸ a 배열의 첫 번째 요소인 a[0]의 주소(&)를 b에 저장한다.

	a[0]	a[1]	a[2]	a[3]	a[4]	← 배열 표기 방법
배열 a	첫 번째	두 번째	세 번째	네 번째	다섯 번째	
	*(a+0)	*(a+1)	*(a+2)	*(a+3)	*(a+4)	← 포인터 표기 방법

183 초치기 Python의 input( ) 함수

- 키보드로 입력받아 변수에 저장하는 함수이다.
- 입력되는 값은 문자열로 취급되어 저장된다.

예 a = input('입력하세요.')
– **입력하세요.**가 출력되고 커서가 깜빡거리며 입력을 기다린다.
– 키보드로 값을 입력하면 변수 a에 저장된다.

184 초치기 Python의 print( ) 함수

인수로 주어진 값을 화면에 출력하는 함수이다.

예 print(82, 24, sep = '–', end = ',')
82와 24 사이에 분리문자 '–'가 출력되고, 마지막에 종료문자 ','가 출력된다.
결과 82–24,

185 초치기 입력 값의 형변환

- input( ) 함수는 입력되는 값을 무조건 문자열로 저장하므로 숫자로 사용하기 위해서는 형을 변환해야 합니다.
- 변환할 데이터가 1개일 때

예 a = int(input( ))
input( )으로 입력받은 값을 정수로 변환하여 변수 a에 저장한다.

- 변환할 데이터가 2개 이상일 때

예 a, b = map(int, input( ).split( ))
input( ).split( )으로 입력받은 2개의 값을 정수로 변환하여 변수 a, b에 저장한다.

186 Python의 리스트(List)

- 리스트는 필요에 따라 개수를 늘이거나 줄일 수 있기 때문에 리스트를 선언할 때 크기를 적지 않는다.
- 배열과 달리 하나의 리스트에 정수, 실수, 문자열 등 다양한 자료형을 섞어서 저장할 수 있다.
- Python에서 리스트의 위치는 0부터 시작한다.

예1 방법1 : a = [10, 'mike', 23.45]
방법2 : a = list([10, 'mike', 23.45])

결과 리스트 a

a[0]	a[1]	a[2]
10	'mike'	23.45

예2 a[0] = 1 → a[0]에 1을 저장한다.

결과 리스트 a

a[0]	a[1]	a[2]
1	'mike'	23.45

187 Python의 딕셔너리(Dictionary)

- 연관된 값을 묶어서 저장하는 용도로 딕셔너리를 사용한다.
- 리스트가 저장된 요소에 접근하기 위한 키로 위치값인 0, 1, 2 등의 숫자를 사용했다면, 딕셔너리에서는 사용자가 원하는 키를 직접 지정한 후 사용한다.
- 딕셔너리에 접근할 때는 딕셔너리 뒤에 대괄호([ ])를 사용하며, 대괄호([ ]) 안에 키를 지정한다.

예1 방법1 : a = {'이름':'홍길동', '나이':25, '주소':'서울'}
방법2 : a = dict({'이름':'홍길동', '나이':25, '주소':'서울'})

결과 리스트 a

a['이름']	a['나이']	a['주소']
'홍길동'	25	'서울'

예2 a['이름'] = '이순신' → 딕셔너리 a의 '이름' 위치에 '이순신'을 저장한다.

결과 리스트 a

a['이름']	a['나이']	a['주소']
'이순신'	25	'서울'

188 Python의 Range

연속된 숫자를 생성하는 것으로, 리스트, 반복문 등에서 많이 사용된다.

예1 a = list(range(5))
0에서 4까지 연속된 숫자를 리스트 a로 저장한다.

리스트 a

0	1	2	3	4

예2 b = list(range(4, 9))
4에서 8까지 연속된 숫자를 리스트 b로 저장한다.

리스트 b

4	5	6	7	8

예3 c = list(range(1, 15, 3))
1에서 14까지 3씩 증가하는 숫자들을 리스트 c로 저장한다.

리스트 c

1	4	7	10	13

189 Python의 슬라이스

문자열이나 리스트와 같은 순차형 객체에서 일부를 잘라(slicing) 반환하는 기능이다.

예 a = ['a', 'b', 'c', 'd', 'e'] 일때
a[1:3] → ['b', 'c']
a[0:5:2] → ['a', 'c', 'e']
a[3:] → ['d', 'e']
a[:3] → ['a', 'b', 'c']
a[::3] → ['a', 'd']

190 Python의 for문

- range를 이용하는 방식

예
```
for i in range(1, 11):
    sum = sum + i
```

i에 1부터 10까지 순서대로 저장하며 sum에 i의 값을 누적시키는 실행문을 반복 수행한다.

- 리스트(List)를 이용하는 방식

예

```
a = [ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ]
for i in a:
    sum = sum + i
```

a 리스트에 저장된 10개의 요소를 i에 순서대로 저장하며 sum에 i의 값을 누적시키는 실행문을 반복 수행한다.

191 Python의 while문

예

```
while i <= 10:
    i = i + 1
```

i가 10보다 작거나 같은 동안 i의 값을 1씩 누적시킨다.

192 Python의 클래스

클래스를 사용하려면 클래스 이름을 정하고 객체 생성을 위한 속성과 메소드(함수)를 정의한 후, 객체를 선언하면 된다.

예제 다음 Python으로 구현된 프로그램의 출력 결과를 확인하시오.

```
   class Cls:
       x = 10
❹      def add(self, a):
❺          return a + self.x
❶  a = Cls( )
❷  a.x = 5
❸❻ print(a.add(5))
```

❶ Cls 클래스의 객체 a를 생성한다.
❷ 객체 a의 변수 x에 5를 저장한다. (a.x = 5)
❸ 5를 인수로 a 객체의 add( ) 메소드를 호출한 후 돌려받은 값을 출력한다.
❹ add( ) 메소드의 시작점이다. ❸번에서 전달받은 5를 a가 받는다.
❺ a와 객체의 변수 x를 더한 값 10을 메소드를 호출했던 ❻번으로 반환한다.
❻ ❺번에서 돌려받은 값 10을 출력한다.

결과 10

193 Python의 클래스 없는 메소드

C언어의 사용자 정의 함수와 같이 클래스 없이 메소드만 단독으로 사용할 수 있다.

예제 다음 Python으로 구현된 프로그램의 출력 결과를 확인하시오.

```
❸ def calc(x, y):
❹     x *= y
❺     return x
❶ a, b = 3, 4
❷ a = calc(a, b)
❻ print(a, b)
```

❶ 변수 a와 b에 3과 4를 저장한다.
❷ a, b 즉 3과 4를 인수로 하여 calc 메소드를 호출한 결과를 a에 저장한다.
❸ calc( ) 메소드의 시작점이다. ❷번에서 전달받은 3과 4를 x와 y가 받는다.
❹ x = x * y이므로 x는 12가 된다.
❺ x의 값을 반환한다. x의 값 12를 ❷번의 a에 저장한다.
❻ 결과 12, 4

194 스크립트 언어의 종류

- 자바스크립트 : 웹 페이지의 동작을 제어하는 데 사용되는 클라이언트용 스크립트 언어
- PHP : Linux, Unix, Windows 운영체제에서 사용 가능한 서버용 스크립트 언어
- 파이썬(Python) : 귀도 반 로섬이 발표한 대화형 인터프리터 언어
- 쉘 스크립트 : 쉘에서 사용되는 명령어들의 조합으로 구성된 스크립트 언어
- Basic : 절차지향 기능을 지원하는 대화형 인터프리터 언어

195 쉘 스크립트에서 사용되는 제어문

- 선택형 : if, case
- 반복형 : for, while, until

196 라이브러리

- 표준 라이브러리 : 프로그래밍 언어에 기본적으로 포함되어 있는 라이브러리로, 여러 종류의 모듈이나 패키지로 구성됨
- 외부 라이브러리 : 개발자들이 필요한 기능들을 만들어 인터넷 등에 공유해 놓은 것으로, 외부 라이브러리를 다운받아 설치한 후 사용함

197 C언어의 stdlib.h

- 자료형 변환, 난수 발생, 메모리 할당에 사용되는 기능들을 제공한다.
- 주요 함수 : atoi, atof, srand, rand, malloc, free 등

198 malloc( ) 함수

- 바이트 단위로 메모리 공간을 동적으로 할당한다.
- 메모리 할당이 불가능할 경우 NULL이 반환된다.

199 JAVA의 예외 처리

- try ~ catch 문을 이용해 예외를 처리한다.
- finally 문은 예외 발생과 관계없이 무조건 수행된다.

200 기억장치의 배치 전략

- 최초 적합(First Fit) : 첫 번째 분할 영역에 배치
- 최적 적합(Best Fit) : 단편화를 가장 작게 남기는 분할 영역에 배치
- 최악 적합(Worst Fit) : 단편화를 가장 많이 남기는 분할 영역에 배치

201 페이지 교체 알고리즘 - FIFO

각 페이지가 주기억장치에 적재될 때마다 그때의 시간을 기억시켜 가장 먼저 들어와서 가장 오래 있었던 페이지를 교체하는 기법이다.

예 다음의 참조 페이지를 세 개의 페이지 프레임을 가진 기억장치에서 FIFO 알고리즘을 사용하여 교체했을 때 페이지 부재의 수는? (단, 초기 페이지 프레임은 모두 비어있는 상태이다.)

참조 페이지	2	3	2	1	5	2	3	5
페이지 프레임	2	2	2	2	5	5	5	5
		3	3	3	3	2	2	2
				1	1	1	3	3
부재 발생	●	●		●	●	●	●	

부재 수 = 6

❶ ❷ ❸

참조 페이지를 각 페이지 프레임에 차례로 적재시키되 이미 적재된 페이지는 해당 위치의 페이지 프레임을 사용한다.

❷ 사용할 페이지 프레임이 없을 경우 가장 먼저 들어와서 오래 있었던 페이지 2를 제거한 후 5를 적재한다.

❸ 그 다음에 적재된 페이지 3을 제거한 후 2를 적재하며, 같은 방법으로 나머지 참조 페이지를 수행한다.

202 스래싱(Thrashing)

프로세스의 처리 시간보다 페이지 교체에 소요되는 시간이 더 많아지는 현상이다.

203 프로세스

- 실행중인 프로그램이다.
- PCB를 가진 프로그램이다.
- 프로세스 내에서의 작업 단위는 스레드이다.

204 스케줄링 - HRN

예 HRN 방식으로 스케줄링할 경우, 입력된 작업이 다음과 같을 때 우선순위가 가장 높은 작업은?

작업	대기시간	서비스시간
A	5	5
B	10	6
C	15	7
D	20	8

- HRN 방식의 우선순위 계산식은 $\frac{\text{대기 시간} + \text{서비스 시간}}{\text{서비스 시간}}$ 이므로 이 식에 대입해서 풀면 된다.
- A 작업은 $\frac{5+5}{5} = 2$
- B 작업은 $\frac{10+6}{6} = 2.6$
- C 작업은 $\frac{15+7}{7} = 3.1$
- D 작업은 $\frac{20+8}{8} = 3.5$이다.
- 결과가 큰 값이 우선순위가 높다.

205 UNIX의 주요 명령어

- fork : 새로운 프로세스를 생성함
- uname : 커널의 버전을 확인함
- wait : 상위 프로세스가 하위 프로세스 종료 등의 event를 기다림
- chmod : 파일의 보호 모드를 설정함
- ls : 현재 디렉터리 내의 파일 목록을 확인함
- cat : 파일 내용을 화면에 표시함
- chown : 소유자를 변경함
- umask
  - 파일이나 디렉터리의 초기 권한을 설정할 때 사용하는 값이다.
  - 파일의 경우 666에서, 디렉터리의 경우 777에서 umask를 뺀 값을 초기 접근 권한으로 갖는다.

206 인터넷 주소 체계 - IPv4

- 8비트씩 4부분, 총 32비트로 구성되어 있다.
- A 클래스에서 E 클래스까지 총 5단계로 구성되어 있다.

207 인터넷 주소 체계 - IPv6

- 16비트씩 8부분, 총 128비트로 구성되어 있다.
- 각 부분을 16진수로 표현하고, 콜론(:)으로 구분한다.
- 주소의 확장성, 융통성, 연동성이 뛰어나고, 품질 보장이 용이하다.
- IPv6 주소 체계 : 유니캐스트(Unicast), 멀티캐스트(Multicast), 애니캐스트(Anycast)

208 OSI 7계층 - 데이터 링크 계층 (Data Link Layer)

- 인접한 개방 시스템들 간에 신뢰성 있고 효율적인 정보 전송을 할 수 있도록 한다.
- 기능 : 흐름 제어, 프레임 동기화, 오류 제어, 순서 제어 등
- 프로토콜 : HDLC, LAPB, PPP, LLC 등

209 OSI 7계층 - 네트워크 계층 (Network Layer)

- 개방 시스템들 간의 네트워크 연결을 관리하고 데이터를 교환 및 중계한다.
- 기능 : 경로 설정, 트래픽 제어, 패킷 정보 전송 등

210 OSI 7계층 - 전송 계층 (Transport Layer)

- 종단 시스템(End-to-End) 간에 투명한 데이터 전송을 가능하게 한다.
- 기능 : 전송 연결 설정, 데이터 전송, 연결 해제 기능, 주소 설정, 다중화, 오류 제어, 흐름 제어 등

211 OSI 7계층 - 세션 계층 (Session Layer)

- 송·수신 측 간의 관련성을 유지하고 대화 제어를 담당한다.
- 대화의 생성, 관리, 종료를 위해 토큰을 사용한다.
- 기능 : 대화 구성 및 동기 제어, 데이터 교환 관리 등

212 네트워크 관련 주요 장비

- 리피터(Repeater) : 원래의 신호 형태로 재생하여 다시 전송함
- 브리지(Bridge) : LAN과 LAN을 연결하거나 LAN 안에서의 컴퓨터 그룹을 연결함
- 라우터(Router) : 데이터 전송의 최적 경로를 선택함
- 스위치(Switch) : LAN과 LAN을 연결하여 훨씬 더 큰 LAN을 만드는 장치
- 브라우터(Brouter) : 브리지와 라우터의 기능을 모두 수행하는 장치로, 브리지 기능은 내부 네트워크를 분리하는 용도로 사용하고 라우터 기능은 외부 네트워크에 연결하는 용도로 사용함
- 전처리기 : 제어 및 검사 등을 미리 수행하여, 컴퓨터의 부담을 줄여주는 역할을 함

213 TCP/IP 프로토콜 - MQTT

발행-구독 기반의 메시징 프로토콜로, IoT 환경에서 자주 사용된다.

214 TCP/IP 프로토콜 - TCP

- 신뢰성 있는 연결형 서비스를 제공한다.
- 양방향 연결형 서비스를 제공한다.
- 기능 : 패킷의 다중화, 순서 제어, 오류 제어, 흐름 제어, 스트림 전송 등

215 TCP/IP 프로토콜 - UDP

- 비연결형 서비스를 제공한다.
- 흐름 제어나 순서 제어가 없어 전송 속도가 빠르다.
- 실시간 전송에 유리하다.

216 TCP/IP 프로토콜 - ARP

호스트의 IP 주소를 호스트와 연결된 네트워크 접속 장치의 물리적 주소(MAC Address)로 바꾼다.

217 CSMA/CD

IEEE 802.3 LAN에서 사용되는 전송 매체 접속 제어(MAC) 방식이다

5과목 정보시스템 구축 관리

218 소프트웨어 재사용의 이점

- 개발 시간과 비용 단축
- 소프트웨어 품질 향상
- 소프트웨어 개발의 생산성 향상
- 시스템 명세, 설계, 코드 등 문서 공유

219 소프트웨어 재공학의 주요 활동

- 분석(Analysis) : 기존 소프트웨어의 명세서를 확인하여 소프트웨어의 동작을 이해하고, 재공학할 대상을 선정하는 활동
- 재구성(Restructuring) : 기존 소프트웨어의 구조를 향상시키기 위하여 코드를 재구성하는 활동
- 역공학(Reverse Engineering) : 기존 소프트웨어를 분석하여 소프트웨어 개발 과정과 데이터 처리 과정을 설명하는 분석 및 설계 정보를 재발견하거나 다시 만들어 내는 활동
- 이식(Migration) : 기존 소프트웨어를 다른 운영체제나 하드웨어 환경에서 사용할 수 있도록 변환하는 활동

220 CASE 사용의 이점

- 소프트웨어 모듈의 재사용성이 향상된다.
- 소프트웨어 개발 기간을 단축하고 개발 비용을 절감할 수 있다.
- 자동화된 기법을 통해 소프트웨어 품질이 향상된다.
- 소프트웨어의 유지보수를 간편하게 수행할 수 있다.

221 비용 산정 기법 - LOC 기법

소프트웨어 각 기능의 원시 코드 라인 수의 비관치, 낙관치, 기대치를 측정하여 예측치를 구하고 이를 이용하여 비용을 산정하는 기법이다.

222 LOC 기법의 산정 공식

- 노력(인월)
  - 개발 기간 × 투입 인원
  - LOC / 1인당 월평균 생산 코드 라인 수
- 개발 비용 : 노력(인월) × 단위 비용(1인당 월평균 인건비)
- 개발 기간 : 노력(인월) / 투입 인원
- 생산성 : LOC / 노력(인월)

223 비용 산정 기법 - COCOMO

보헴(Boehm)이 제안한 것으로, 원시 프로그램의 규모(LOC)에 의한 비용 산정 기법이다.

224 COCOMO의 소프트웨어 개발 유형

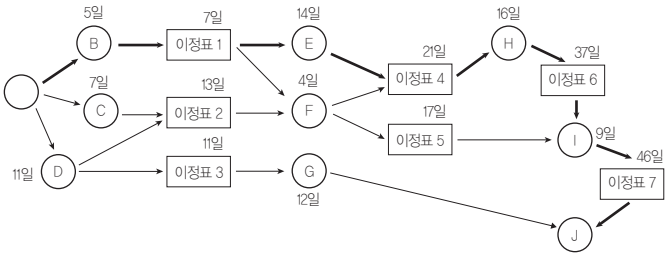
- 조직형(Organic Mode) : 기관 내부에서 개발된 중·소 규모의 소프트웨어로 일괄 자료 처리나 과학기술 계산용, 비즈니스 자료 처리용으로 5만(50KDSI) 라인 이하의 소프트웨어를 개발하는 유형
- 반분리형(Semi-Detached Mode) : 조직형과 내장형의 중간형으로 트랜잭션 처리 시스템이나 운영체제, 데이터베이스 관리 시스템 등의 30만(300KDSI) 라인 이하의 소프트웨어를 개발하는 유형
- 내장형(Embedded Mode) : 내장형은 초대형 규모의 트랜잭션 처리 시스템이나 운영체제 등의 30만(300KDSI) 라인 이상의 소프트웨어를 개발하는 유형

225 PERT (프로그램 평가 및 검토 기술)

- 각 작업별로 낙관적인 경우, 가능성이 있는 경우, 비관적인 경우로 나누어 각 단계별 종료 시기를 결정하는 방법이다.
- 결정 경로, 작업에 대한 경계 시간, 작업 간의 상호 관련성 등을 알 수 있다.

226 임계 경로 소요 기일

임계 경로는 최장 경로(굵은 선)를 의미한다.



227 간트 차트(Gantt Chart)

- 프로젝트의 작업 일정을 막대 도표를 이용하여 표시하는 프로젝트 일정표이다.
- 수평 막대의 길이는 각 작업(Task)의 기간을 나타낸다.

228 소프트웨어 프로젝트 관리

주어진 기간 내에 최소의 비용으로 사용자를 만족시키는 시스템을 개발하기 위한 전반적인 활동을 의미한다.

229 위험 관리(Risk Analysis)

- 프로젝트 추진 과정에서 예상되는 각종 돌발 상황(위험)을 미리 예상하고 이에 대한 적절한 대책을 수립하는 일련의 활동을 의미한다.
- 위험 관리 절차 : 위험 식별 → 위험 분석 및 평가 → 위험 관리 계획 → 위험 감시 및 조치
- 위험 감시(Risk Monitoring) : 위험 요소 징후들에 대하여 계속적으로 인지하는 것

230 CMMI의 개념

소프트웨어 개발 조직의 업무 능력 및 조직의 성숙도를 평가하는 모델이다.

231 CMMI의 소프트웨어 프로세스 성숙도 5단계

- 초기(Initial)
- 관리(Managed)
- 정의(Defined)
- 정량적 관리(Quantitatively Managed)
- 최적화(Optimizing)

232 SDN(Software Defined Networking)

네트워크를 컴퓨터처럼 모델링하여 여러 사용자가 각각의 소프트웨어들로 네트워킹을 가상화하여 제어하고 관리하는 네트워크이다.

233 유비쿼터스 컴퓨팅

언제 어디서나 어떤 기기를 통해서도 컴퓨팅이 가능한 환경을 의미하는 포괄적인 개념이다.

234 클라우드 컴퓨팅의 서비스 유형

- IaaS
- PaaS
- SaaS

235 서버 팜

여러 대의 서버를 집약하여 하나의 시스템처럼 운영함으로써 대용량 데이터 처리와 안정적인 서비스를 제공하는 형태의 설비이다.

236 웹 크롤링

- 웹에서 자동화된 프로그램을 이용해 데이터를 수집하는 작업이다.
- 지원 프레임워크 : 파이썬, 스크래피

237 메타버스

현실 세계와 같은 사회·경제·문화 활동이 이뤄지는 3차원 가상 세계를 가리킨다.

238 메시 네트워크 (Mesh Network)

수십에서 수천 개의 디바이스를 그물망(Mesh)과 같이 유기적으로 연결하여 모든 구간을 동일한 무선망처럼 구성하여 사용자가 안정적인 네트워크를 사용할 수 있게 한다.

239 클라우드 기반 HSM

- 클라우드를 기반으로 암호화 키의 생성·저장·처리 등의 작업을 수행하는 보안기기를 가리키는 용어이다.
- 암호화 키 생성이 하드웨어적으로 구현되기 때문에 소프트웨어적으로 구현된 암호 기술이 가지는 보안 취약점을 무시할 수 있다.

240 파스-타(PaaS-TA)

소프트웨어 개발 환경을 제공하기 위해 개발한 개방형 클라우드 컴퓨팅 플랫폼이다.

241 징(Zing)

10cm 이내 거리에서 3.5Gbps 속도의 데이터 전송이 가능한 초고속 근접무선통신(NFC)이다.

242 SSO(Single Sign On)

한 번의 로그인으로 개인이 가입한 모든 사이트를 이용할 수 있게 해주는 시스템이다.

243 WPA (Wi-Fi Protected Access)

Wi-Fi에서 제정한 무선 랜(WLAN) 인증 및 암호화 관련 표준이다.

244 RIP(Routing Information Protocol)

- IGP에 속하는 라우팅 프로토콜로, 거리 벡터 라우팅 프로토콜이라고도 불린다.
- 최대 홉(Hop) 수를 15로 제한한다.

245 OSPF(Open Shortest Path First protocol)

라우팅 정보에 노드 간의 거리 정보, 링크 상태 정보를 실시간으로 반영하여 최단 경로로 라우팅을 지원하는 프로토콜

246 증발품(Vaporware)

판매 계획 또는 배포 계획은 발표되었으나 실제로 고객에게 판매되거나 배포되지 않고 있는 소프트웨어이다.

247 도커(Docker)

컨테이너 기술을 자동화하여 쉽게 사용할 수 있게 하는 오픈소스 프로젝트이다.

248 매시업(Mashup)

웹에서 제공하는 정보 및 서비스를 이용하여 새로운 소프트웨어나 서비스, 데이터베이스 등을 만드는 기술이다.

249 디지털 트윈(Digital Twin)

현실 속의 사물을 소프트웨어로 가상화한 모델이다.

250 서비스형 블록체인(BaaS)

블록체인(Blockchain) 앱의 개발 환경을 클라우드 기반으로 제공하는 서비스이다.

251 TCP 래퍼(TCP Wrapper)

외부 컴퓨터의 접속 인가 여부를 점검하여 접속을 허용 및 거부하는 보안용 도구이다.

252 DPI(Deep Packet Inspection)

OSI 7 Layer 전 계층의 프로토콜과 패킷 내부의 콘텐츠를 파악하여 침입 시도, 해킹 등을 탐지하고, 트래픽을 조정하기 위한 패킷 분석 기술이다.

253 허니팟(Honeypot)

비정상적인 접근의 탐지를 위해 의도적으로 설치해 둔 시스템이다.

254 고가용성 솔루션(HACMP)

긴 시간 동안 안정적인 서비스 운영을 위해 장애 발생 시 즉시 다른 시스템으로 대체 가능한 환경을 구축하는 메커니즘을 의미한다.

255 하둡(Hadoop)

- 오픈 소스를 기반으로 한 분산 컴퓨팅 플랫폼이다.
- 하둡과 관계형 데이터베이스(RDB) 간 대용량 데이터를 전송할 때 스쿱(Sqoop)이라는 도구를 이용한다.

256 맵리듀스(MapReduce)

대용량 데이터를 분산 처리하기 위한 목적으로 개발된 프로그래밍 모델이다.

257 데이터 마이닝(Data Mining)

데이터 웨어하우스에 저장된 대량의 데이터 집합에서 사용자의 요구에 따라 유용하고 가능성 있는 정보를 발견하기 위한 기법이다.

258 OLAP(Online Analytical Processing)

- 다차원으로 이루어진 데이터로부터 통계적인 요약 정보를 분석하여 의사결정에 활용하는 방식을 말한다.
- OLAP 연산 : Roll-up, Drill-down, Drill-through, Drill-across, Pivoting, Slicing, Dicing

259 회복(Recovery)

트랜잭션들을 수행하는 도중 장애가 발생하여 데이터베이스가 손상되었을 때 손상되기 이전의 정상 상태로 복구하는 작업이다.

260 즉각 갱신 기법 (Immediate Update)

- 트랜잭션이 데이터를 갱신하면 트랜잭션이 부분 완료되기 전이라도 즉시 실제 데이터베이스에 반영하는 방법이다.
- 장애가 발생하여 회복 작업할 경우를 대비하여 갱신된 내용들은 Log에 보관시킨다.

261 병행제어의 목적

- 데이터베이스의 공유를 최대화한다.
- 시스템의 활용도를 최대화한다.
- 데이터베이스의 일관성을 유지한다.
- 사용자에 대한 응답 시간을 최소화한다.

262 로킹 단위 (Locking Granularity)

- 병행제어에서 한꺼번에 로킹할 수 있는 객체의 크기를 의미한다.
- 로킹 단위가 크면 로크 수가 작아 관리하기 쉽지만 병행성 수준이 낮아진다.
- 로킹 단위가 작으면 로크 수가 많아 관리하기 복잡해 오버헤드가 증가하지만 병행성 수준이 높아진다.

263 교착상태 발생의 필요 충분 조건 4가지

- 상호 배제(Mutual Exclusion)
- 점유와 대기(Hold and Wait)
- 비선점(Non-preemption)
- 환형 대기(Circular Wait)

264 회피 기법(Avoidance)

- 교착상태가 발생할 가능성을 배제하지 않고 교착상태가 발생하면 적절히 피해 나가는 방법으로, 주로 은행원 알고리즘(Banker's Algorithm)이 사용된다.
- 은행원 알고리즘(Banker's Algorithm) : E. J. Dijkstra가 제안한 것으로, 은행에서 모든 고객의 요구가 충족되도록 현금을 할당하는 데서 유래한 기법

265 보안 3대 요소

- 기밀성 : 시스템 내의 정보와 자원은 인가된 사용자에게만 접근이 허용되며, 정보가 전송 중에 노출되더라도 데이터를 읽을 수 없음
- 무결성 : 시스템 내의 정보는 오직 인가된 사용자만 수정할 수 있음
- 가용성 : 인가받은 사용자는 언제라도 사용할 수 있음

266 세션 하이재킹 (Session Hijacking)

- 서버에 접속하고 있는 클라이언트들의 세션 정보를 가로채는 공격기법으로, 세션 가로채기라고도 한다.
- 탐지 방법 : 비동기화 상태 탐지, ACK Storm 탐지, 패킷의 유실 탐지, 예상치 못한 접속의 리셋 탐지 등

267 SQL 삽입(SQL Injection)

웹 응용 프로그램에 SQL을 삽입하여 내부 데이터베이스(DB) 서버의 데이터를 유출 및 변조하고, 관리자 인증을 우회하는 보안 약점이다.

268 경로 조작 및 자원 삽입

데이터 입출력 경로를 조작하여 서버 자원을 수정·삭제할 수 있는 보안 약점이다.

269 크로스사이트 스크립팅(XSS; Cross Site Scripting)

웹페이지에 악의적인 스크립트를 삽입하여 방문자들의 정보를 탈취하거나, 비정상적인 기능 수행을 유발하는 보안 약점이다.

270 스택 가드(Stack Guard)

메모리상에서 프로그램의 복귀 주소와 변수 사이에 특정 값을 저장한 후 그 값이 변경되었을 경우 오버플로우 상태로 판단하여 프로그램 실행을 중단함으로써 잘못된 복귀 주소의 호출을 막는 기술이다.

271 접근 지정자(접근 제어자)

- 프로그래밍 언어에서 특정 개체를 선언할 때 외부로부터의 접근을 제한하기 위해 사용되는 예약어이다.
- 종류 : Public, Protected, Default, Private

272 개인키 암호화(Private Key Encryption) 기법

- 동일한 키로 데이터를 암호화하고 복호화한다.
- 알고리즘이 단순하므로 암호화/복호화 속도가 빠르다.

273 개인키 암호화 기법의 종류

- 블록 암호화 방식
  - 한 번에 하나의 데이터 블록을 암호화하는 방식
  - 종류 : DES, SEED, AES, ARIA, IDEA 등
- 스트림 암호화 방식
  - 평문과 동일한 길이의 스트림을 생성하여 비트/바이트/워드 단위로 암호화하는 방식
  - 종류 : LFSR, RC4 등

274 해시(Hash)

- 임의의 길이의 입력 데이터나 메시지를 고정된 길이의 값이나 키로 변환하는 것을 의미한다.
- 복호화가 거의 불가능한 일방향 함수에 해당한다.
- 해시 함수의 종류 : SHA 시리즈, MD4, MD5, N-NASH, SNEFRU 등

275 공개키 암호화 (Public Key Encryption) 기법

- 데이터를 암호화할 때 사용하는 공개키(Public Key)는 데이터베이스 사용자에게 공개하고, 복호화할 때의 비밀키(Secret Key)는 관리자가 비밀리에 관리한다.
- 암호화 대상이 n개일 때 사용되는 키의 개수는 2n이다.
- 대표적으로 RSA(Rivest Shamir Adleman)가 있다.

276 양방향 알고리즘 종류

- DES(Data ncryption Standard) : 1975년 미국 NBS에서 발표한 64비트의 개인키 암호화 알고리즘
- AES(Advanced Encryption Standard) : DES에 한계를 느낀 미국 표준 기술 연구소(NIST)가 2001년 발표한 개인키 암호화 알고리즘
- RSA(Rivest Shamir Adleman) : 큰 숫자를 소인수분해 하기 어렵다는 것에 기반하여 만들어진 공개키 암호화 알고리즘
- ECC : 이산대수 문제를 이용해 기밀성과 효율성을 높인 암호화 알고리즘
- Rabin : 소인수분해의 어려움에 안전성의 근거를 둔 암호화 알고리즘

277 솔트(Salt)

암호화를 수행하기에 앞서 원문에 무작위의 값을 덧붙이는 과정이다.

278 죽음의 핑(Ping of Death)

Ping 명령을 전송할 때 ICMP 패킷의 크기를 인터넷 프로토콜 허용 범위 이상으로 전송하여 공격 대상의 네트워크를 마비시키는 서비스 거부 공격 방법이다.

279 SMURFING(스머핑)

IP나 ICMP의 특성을 악용하여 엄청난 양의 데이터를 한 사이트에 집중적으로 보냄으로써 네트워크를 불능 상태로 만드는 공격 방법이다.

280 Land

패킷을 전송할 때 송신 IP 주소와 수신 IP 주소를 모두 공격 대상의 IP 주소로 하여 공격 대상에게 전송하는 공격 기법이다.

281 DDoS(Distributed Denial of Service, 분산 서비스 거부) 공격

- 여러 곳에 분산된 공격 지점에서 한 곳의 서버에 대해 분산 서비스 공격을 수행하는 것이다.
- 분산 서비스 공격용 툴의 종류 : Trin00, TFN(Tribe Flooding Network), TFN2K, Stacheldraht 등

282 파밍(Pharming)

정상적인 사이트에 접속해도 도메인을 탈취하여 가짜 사이트로 접속하게 한 다음 개인 정보나 금융정보를 몰래 빼내는 행위이다.

283 웜(Worm)

네트워크를 통해 연속적으로 자신을 복제하여 시스템의 부하를 높임으로써 결국 시스템을 다운시키는 바이러스의 일종이다.

284 랜섬웨어(Ransomware)

인터넷 사용자의 컴퓨터에 잠입해 내부 문서나 파일 등을 암호화해 사용자가 열지 못하게 하는 프로그램으로, 암호 해독용 프로그램의 전달을 조건으로 사용자에게 돈을 요구하기도 한다.

285 인증(認證, Authentication)

- 로그인을 요청한 사용자의 정보를 확인하고 접근 권한을 검증하는 보안 절차이다.
- 인증의 유형
  - 지식 기반 인증(Something You Know) : 고정된 패스워드, 아이핀 등
  - 소유 기반 인증(Something You Have) : 신분증, OTP, 스마트카드 등
  - 행위 기반 인증(Something You Do) : 서명, 움직임, 음성 등

286 커널 로그의 종류

- wtmp : 성공한 로그인/로그아웃과 시스템의 시작/종료 시간에 대한 로그
- utmp : 현재 로그인한 사용자의 상태에 대한 로그
- btmp : 실패한 로그인에 대한 로그
- lastlog : 마지막으로 성공한 로그인에 대한 로그

287 침입 탐지 시스템(IDS; Intrusion Detection System)

- 컴퓨터 시스템의 비정상적인 사용, 오용, 남용 등을 실시간으로 탐지하는 시스템이다.
- 오용 탐지(Misuse Detection) : 미리 입력해 둔 공격 패턴이 감지되면 이를 알려줌
- 이상 탐지(Anomaly Detection) : 평균적인 시스템의 상태를 기준으로 비정상적인 행위나 자원의 사용이 감지되면 이를 알려줌

288 SSH(Secure SHell, 시큐어 셸)

- 다른 컴퓨터에 로그인, 원격 명령 실행, 파일 복사 등을 수행할 수 있도록 다양한 기능을 지원하는 프로토콜 또는 이를 이용한 응용 프로그램이다.
- 기본적으로는 22번 포트를 사용한다.